

Preparing and Analyzing Survey Data in R

George M. Harrison University of Hawai'i at Mānoa, College of Education

April, 2021

Contents

1	Introduction	2
1.1	Prior knowledge	2
2	Preparing the Data	2
2.1	Import the data from the CSV file	2
2.2	Managing the structure of the data	4
2.3	Recoding response-category data	6
2.4	Creating Composite Scores (AKA Indexes)	12
2.5	A general formula for rescaling data	13
2.6	Create and use a custom function	14
2.7	The code in one place for one of the composite scores	16
2.8	A shortcut, when all of the items are on the same scale	16
2.9	Summary	17
3	Installing and Using Packages	17
3.1	Seeing if we need to install a package	17
3.2	Installing a package	17
3.3	Using a function from a package	18
4	Estimating Descriptive Statistics	19
4.1	Descriptive statistics of categorical data	19
4.2	Descriptive statistics of interval-level data	26
4.3	But really, it's inferential	31
5	Descriptive Statistics by Group	31
5.1	Using the <code>describeBy()</code> function	31
5.2	Confidence intervals for each group's estimated mean	32
5.3	Plotting the by-group means and confidence intervals	35

6	Internal Consistency Reliability	39
6.1	Doing it manually	39
6.2	Doing it with the psych package	40
6.3	Summary	42
7	Introduction to the Analysis of Complex Survey Data	43
7.1	Packages	43
7.2	Example data	43
7.3	The three hypothetical complex designs	44
7.4	Cluster-sampled design	44
7.5	Stratified design	49
7.6	Stratified cluster-sampled design	51
7.7	A simple random sample design, for comparison	53

1 Introduction

This site provides guidance in preparing and analyzing survey data in R. We begin with a chapter on preparing data and managing packages, then practice estimating descriptive statistics and internal-consistency reliability coefficients. We will eventually end with an example of how to analyze complex survey data, such as that collected from clustered and stratified samples.

Along the way, there are explanations about how to use R, including the use of functions and packages. One way to study using this guide is to follow along with your own RStudio file opened while you try out the code. Copying and pasting is one way. You can keep notes for yourself in your R code by preceding your annotations with the hash symbol (#).

The final assignment will ask you to practice much of what is in this tutorial. It is also wise to keep your R code files for future references when you do use it analyze your own data.

1.1 Prior knowledge

Before going through these tutorials, it is necessary to have installed R and RStudio. Also, it is highly recommended that you practice some of the basics, such as those in the introductory tutorial at http://www2.hawaii.edu/~georgeha/surveys/_book/

2 Preparing the Data

In this section, we are using R to prepare data for analysis. We will use mock data from the TIMSS Teacher Questionnaire.

2.1 Import the data from the CSV file

I generated these mock data as fake responses to the TIMSS math teacher questionnaire https://nces.ed.gov/timss/pdf/2015_8th_grade_Teacher_Questionnaire_Math.pdf. You can also get real data from their site. We could use the code below with those data (though we would need to address missing values). In

other words, this can be part of our data analysis plan and we created data using our best guess about how people would respond. Then, we assume it functions well, and then analyze the real data.

If you've opened this RStudio file from the same folder that contains the CSV file, `Week13_Timss_Mock_Raw.csv`, your working directory will be that location and you can import data using this code:

```
timss <- read.csv('Week13_Timss_Mock_Raw.csv')
```

If you prefer to download the data, it will be temporarily available here:

```
timss <- read.csv("http://www2.hawaii.edu/~georgeha/Handouts/Week13_Timss_Mock_Raw.csv")
```

2.1.1 Examining the imported data set

We can print the first and last five rows of data using the `head()` and `tail()` functions:

```
head(timss)
tail(timss)
```

We can also select which rows to print using indexing. Here, it prints only rows 1, 3, 5, 7, and 9:

```
timss[c(1,3,5,7,9), ]
```

We can also print the data set to console or view it in a new tab.

```
timss
View(timss)
```

Let's check that the data set has the number of observations we expect. We should have 30 mock teachers. This prints the number of rows in the column labeled TID.

```
length(timss$TID)
```

```
## [1] 30
```

Good. There are 30. We can also see what how many rows and columns there are with the `dim()` function, which prints the number of rows then the number of columns.

```
dim(timss)
```

```
## [1] 30 31
```

Let's see what the names of the variables are:

```
names(timss)
```

```
## [1] "TID"           "Q4_Age"         "Q5_Degree"      "Q6_Math"
## [5] "Q6_Bio"        "Q6_Phys"        "Q6_Chem"        "Q6_EarthS"
## [9] "Q6_EdMat"      "Q6_EdSci"       "Q6_EdGen"       "Q6_Othr"
## [13] "Q13_NumSs"     "Q15_StCentrd_a" "Q15_StCentrd_b" "Q15_StCentrd_c"
## [17] "Q15_StCentrd_d" "Q15_StCentrd_e" "Q15_StCentrd_f" "Q15_StCentrd_g"
## [21] "Q16_Lims_a"    "Q16_Lims_b"     "Q16_Lims_c"     "Q16_Lims_d"
## [25] "Q16_Lims_e"    "Q16_Lims_f"     "Q16_Lims_g"     "Q17_Mins"
## [29] "Q19_Didac_a"   "Q19_Didac_b"    "Q19_Didac_c"
```

If we enclose this code in the `dput()` function, we can then copy and paste names into code.

```
dput(names(timss))
```

```
## c("TID", "Q4_Age", "Q5_Degree", "Q6_Math", "Q6_Bio", "Q6_Phys",  
## "Q6_Chem", "Q6_EarthS", "Q6_EdMat", "Q6_EdSci", "Q6_EdGen", "Q6_Othr",  
## "Q13_NumSs", "Q15_StCentrd_a", "Q15_StCentrd_b", "Q15_StCentrd_c",  
## "Q15_StCentrd_d", "Q15_StCentrd_e", "Q15_StCentrd_f", "Q15_StCentrd_g",  
## "Q16_Lims_a", "Q16_Lims_b", "Q16_Lims_c", "Q16_Lims_d", "Q16_Lims_e",  
## "Q16_Lims_f", "Q16_Lims_g", "Q17_Mins", "Q19_Didac_a", "Q19_Didac_b",  
## "Q19_Didac_c")
```

This will be handy if we wish to copy and paste a string of column names from our console into our code because it formats it with quotes around the names and commas in between them, which is what we want to use in the `c()` function.

2.2 Managing the structure of the data

Let's look at the structure of our data set, using the `str()` function.

```
str(timss)
```

```
## 'data.frame':   30 obs. of  31 variables:  
## $ TID           : int  1 2 3 4 5 6 7 8 9 10 ...  
## $ Q4_Age        : num  2 3 3 2 5 6 5 2 4 3 ...  
## $ Q5_Degree     : num  4 4 4 5 3 4 5 4 4 4 ...  
## $ Q6_Math       : chr  "No" "No" "No" "No" ...  
## $ Q6_Bio        : chr  "No" "No" "No" "No" ...  
## $ Q6_Phys       : chr  "No" "No" "Yes" "No" ...  
## $ Q6_Chem       : chr  "No" "No" "No" "No" ...  
## $ Q6_EarthS     : chr  "No" "No" "No" "No" ...  
## $ Q6_EdMat      : chr  "No" "No" "No" "Yes" ...  
## $ Q6_EdSci      : chr  "No" "No" "No" "No" ...  
## $ Q6_EdGen      : chr  "Yes" "Yes" "No" "No" ...  
## $ Q6_Othr       : chr  "No" "No" "Yes" "No" ...  
## $ Q13_NumSs     : int  8 12 19 16 18 26 29 16 30 22 ...  
## $ Q15_StCentrd_a: int  2 3 3 1 2 3 4 1 4 3 ...  
## $ Q15_StCentrd_b: int  2 1 2 1 1 3 2 1 2 2 ...  
## $ Q15_StCentrd_c: int  1 4 2 2 3 3 2 1 2 2 ...  
## $ Q15_StCentrd_d: int  1 2 3 2 2 3 3 1 4 4 ...  
## $ Q15_StCentrd_e: int  2 2 3 1 2 3 4 2 2 2 ...  
## $ Q15_StCentrd_f: int  3 3 3 1 3 4 2 1 1 4 ...  
## $ Q15_StCentrd_g: int  1 2 3 1 2 3 3 1 3 3 ...  
## $ Q16_Lims_a    : int  2 1 2 1 1 2 3 1 3 1 ...  
## $ Q16_Lims_b    : int  1 1 2 2 1 1 2 1 3 2 ...  
## $ Q16_Lims_c    : int  3 3 3 3 3 3 3 2 2 2 ...  
## $ Q16_Lims_d    : int  2 2 2 2 2 2 2 1 2 3 ...  
## $ Q16_Lims_e    : int  1 1 2 2 1 2 2 2 2 2 ...  
## $ Q16_Lims_f    : int  1 1 1 1 1 1 1 1 2 1 ...  
## $ Q16_Lims_g    : int  2 1 2 1 1 1 1 2 1 1 ...  
## $ Q17_Mins      : int  160 180 190 200 210 180 200 240 150 160 ...  
## $ Q19_Didac_a   : int  3 4 2 3 3 3 2 3 2 1 ...
```

```
## $ Q19_Didac_b : int 3 2 1 4 1 4 2 2 2 2 ...
## $ Q19_Didac_c : int 4 3 2 3 2 2 2 1 2 1 ...
```

We can see that Age and Degree are numeric. We see that the Q6 demographic questions are “character”, which is appropriate for data that are text in CSV file. The other variables are numeric (integer).¹

Let’s change a character variable to numeric data. For the Q6 variables, there are “No” and “Yes” responses. These are **not** numbers, so we cannot perform mathematical functions on them. We can create a new dichotomous variable from these one or more nominal variables, just like how we did in Excel. Let’s create a `Major_in_Educ` variable using the `ifelse()` function. The vertical bar, |, symbolizes “or”, so the following code is *if EdMath is Yes, or if EdScience is Yes, or if EdGen is Yes, code the new variable as 1; otherwise, code it as 0*.

```
timss$Q6_Major_in_Edu <- ifelse(timss$Q6_EdMat == "Yes" |
                               timss$Q6_EdSci == "Yes" |
                               timss$Q6_EdGen == "Yes", 1, 0)
```

Let’s verify it worked by examining some rows of data—here, the first 12 rows and the columns that were used to get our new variable as well as the new variable:

```
timss[1:12, c("TID", "Q6_EdMat", "Q6_EdSci", "Q6_EdGen", "Q6_Major_in_Edu")]
```

```
##      TID Q6_EdMat Q6_EdSci Q6_EdGen Q6_Major_in_Edu
## 1      1      No      No      Yes      1
## 2      2      No      No      Yes      1
## 3      3      No      No      No       0
## 4      4      Yes     No      No       1
## 5      5      Yes     No      No       1
## 6      6      No      No      Yes      1
## 7      7      No      No      No       0
## 8      8      Yes     No      No       1
## 9      9      No      No      No       0
## 10     10     No      No      No       0
## 11     11     No      Yes     No       1
## 12     12     No      No      No       0
```

The variable we just created is in the rightmost column. We can look at the structure of this selected set of columns and see that the last one is numeric.

```
str(timss[, c("Q6_EdMat", "Q6_EdSci", "Q6_EdGen", "Q6_Major_in_Edu")])
```

```
## 'data.frame': 30 obs. of 4 variables:
## $ Q6_EdMat : chr "No" "No" "No" "Yes" ...
## $ Q6_EdSci : chr "No" "No" "No" "No" ...
## $ Q6_EdGen : chr "Yes" "Yes" "No" "No" ...
## $ Q6_Major_in_Edu: num 1 1 0 1 1 1 0 1 0 0 ...
```

This dummy coding is valuable for analysis. We have prepared this new variable so it can serve as a predictor in a regression model (i.e., a general linear model).

¹In earlier versions of R, the `read.csv()` function read these as factors, which are appropriate for nominal and ordinal level data.

2.3 Recoding response-category data

2.3.1 Reverse-coding and rescaling data

Questions 15a–15g have numeric codes that are not logical. For instance, *Every or almost every lesson* is 1 but *Never* is coded as 4. Usually, we use higher values for more of a construct and lower values for less.

So, let's change our coding so that *Never* is coded as 0, *Some lessons* is 1, *About half the lessons* is 2, and *Every or almost every lesson* is 3. So, we are going from a 4-to-1 scale to a 0-to-3 scale.

The general formula for reverse coding is this:

- $\text{newvalue} = (\text{num of response options}) + (\text{bottom value of new scale}) - (\text{observed value in the original scale})$

So with our data in Questions 15a–15g,

- How many response categories are there? *Answer:* 4
- What's the bottom value on our NEW scale going to be? *Answer:* 0
- What are our original data? *Answer:* all of the data in Q15a through Q15g

Maybe we forgot the names of the variables we were investigating. Let's use `names()` to see what's available.

```
names(timss)
```

```
## [1] "TID"           "Q4_Age"         "Q5_Degree"      "Q6_Math"
## [5] "Q6_Bio"        "Q6_Phys"        "Q6_Chem"        "Q6_EarthS"
## [9] "Q6_EdMat"      "Q6_EdSci"       "Q6_EdGen"       "Q6_Othr"
## [13] "Q13_NumSs"     "Q15_StCentrd_a" "Q15_StCentrd_b" "Q15_StCentrd_c"
## [17] "Q15_StCentrd_d" "Q15_StCentrd_e" "Q15_StCentrd_f" "Q15_StCentrd_g"
## [21] "Q16_Lims_a"    "Q16_Lims_b"     "Q16_Lims_c"     "Q16_Lims_d"
## [25] "Q16_Lims_e"    "Q16_Lims_f"     "Q16_Lims_g"     "Q17_Mins"
## [29] "Q19_Didac_a"   "Q19_Didac_b"    "Q19_Didac_c"    "Q6_Major_in_Edu"
```

Let's try the calculation on a single variable and add "R" to the new variable name so we can remember this is the reversed version:

```
timss$Q15_StCentrd_aR <- 4 + 0 - timss$Q15_StCentrd_a
```

Let's verify it worked by comparing the reversed and the original data with first 10 rows. Any old 4s should now be 0 (because $4 + 0 - 4 = 0$), 3s should be 1, 2s should still be 2, and 1s should be 3.

```
timss[1:10, c("Q15_StCentrd_a", "Q15_StCentrd_aR")]
```

```
##      Q15_StCentrd_a Q15_StCentrd_aR
## 1                2                2
## 2                3                1
## 3                3                1
## 4                1                3
## 5                2                2
## 6                3                1
## 7                4                0
## 8                1                3
## 9                4                0
## 10               3                1
```

Let's do this same operation with the remaining 6 items in Q15 (Q15b through Q15g). We could repeat the line above with the new variable names in each line. Or, we can create an R object that contains the names of these 7 variables and then perform the same calculation on this object:

Step 1. Let's create an object that contains the 7 items in Question 15. We'll call this Q15.²

```
Q15 <- c("Q15_StCentrd_a", "Q15_StCentrd_b", "Q15_StCentrd_c", "Q15_StCentrd_d",
        "Q15_StCentrd_e", "Q15_StCentrd_f", "Q15_StCentrd_g")
```

This object, Q15, is only a list of names, but we can use it to refer to those data.³

```
Q15
```

```
## [1] "Q15_StCentrd_a" "Q15_StCentrd_b" "Q15_StCentrd_c" "Q15_StCentrd_d"
## [5] "Q15_StCentrd_e" "Q15_StCentrd_f" "Q15_StCentrd_g"
```

Let's use our object, now: Here are the first five people's responses on these items:

```
timss[1:5, Q15]
```

```
##   Q15_StCentrd_a Q15_StCentrd_b Q15_StCentrd_c Q15_StCentrd_d Q15_StCentrd_e
## 1              2              2              1              1              2
## 2              3              1              4              2              2
## 3              3              2              2              3              3
## 4              1              1              2              2              1
## 5              2              1              3              2              2
##   Q15_StCentrd_f Q15_StCentrd_g
## 1              3              1
## 2              3              2
## 3              3              3
## 4              1              1
## 5              3              2
```

Let's check the range of the original data. The minimum should be 1 and the maximum 4:

```
range(timss[, Q15])
```

```
## [1] 1 4
```

So, we have used the object Q15 in two different functions. This is better than retyping this long string: `c("Q15_StCentrd_a", "Q15_StCentrd_b", "Q15_StCentrd_c", "Q15_StCentrd_d", "Q15_StCentrd_e", "Q15_StCentrd_f", "Q15_StCentrd_g")`.

2.3.1.1 Reverse coding Step 2. Let's do the same thing with the new set of variables that we will create. Let's use an "R" to stand for Reversed. We could write out all the names, with R after each variable, or we can take a shortcut and use the `paste0()` function to concatenate an "R" at the end of every element in the Q15 object:

²We can shorten this code using the `paste0()` function and the first 7 letters of the alphabet: `Q15 <- paste0("Q15_StCentrd_", letters[1:7])`

³The benefit here is that we can re-use this object, Q15, in other code to reference these columns.

```
Q15R <- paste0(Q15, "R")
```

Verify that it created the set of names we want:

```
Q15R
```

```
## [1] "Q15_StCentrd_aR" "Q15_StCentrd_bR" "Q15_StCentrd_cR" "Q15_StCentrd_dR"
## [5] "Q15_StCentrd_eR" "Q15_StCentrd_fR" "Q15_StCentrd_gR"
```

Let's create these seven new reverse-coded variables using our formula and these two objects, Q15 and Q15R. Because each object includes the same number of names, it performs the calculation on the first variable, then the next one and so forth.

```
timss[,Q15R] <- (4 + 0 - timss[,Q15])
```

Take a look at the first 6 rows:

```
timss[1:6, Q15]
```

```
##   Q15_StCentrd_a Q15_StCentrd_b Q15_StCentrd_c Q15_StCentrd_d Q15_StCentrd_e
## 1              2              2              1              1              2
## 2              3              1              4              2              2
## 3              3              2              2              3              3
## 4              1              1              2              2              1
## 5              2              1              3              2              2
## 6              3              3              3              3              3
##   Q15_StCentrd_f Q15_StCentrd_g
## 1              3              1
## 2              3              2
## 3              3              3
## 4              1              1
## 5              3              2
## 6              4              3
```

```
timss[1:6, Q15R]
```

```
##   Q15_StCentrd_aR Q15_StCentrd_bR Q15_StCentrd_cR Q15_StCentrd_dR
## 1              2              2              3              3
## 2              1              3              0              2
## 3              1              2              2              1
## 4              3              3              2              2
## 5              2              3              1              2
## 6              1              1              1              1
##   Q15_StCentrd_eR Q15_StCentrd_fR Q15_StCentrd_gR
## 1              2              1              3
## 2              2              1              2
## 3              1              1              1
## 4              3              3              3
## 5              2              1              2
## 6              1              0              1
```

These match our Excel calculations.

Let's look at the first variable side by side:

```
timss[1:6, c(Q15[1], Q15R[1]) ]
```

```
##    Q15_StCentrd_a Q15_StCentrd_aR
## 1                2                2
## 2                3                1
## 3                3                1
## 4                1                3
## 5                2                2
## 6                3                1
```

We used the index [1] to ask for the first element of the Q15 and Q15R objects. This is the same as if we asked for the names:

```
timss[1:6, c("Q15_StCentrd_a", "Q15_StCentrd_aR") ]
```

```
##    Q15_StCentrd_a Q15_StCentrd_aR
## 1                2                2
## 2                3                1
## 3                3                1
## 4                1                3
## 5                2                2
## 6                3                1
```

We could ask for the second element after making a small change to the code:

```
timss[1:6, c(Q15[2], Q15R[2]) ]
```

Let's see whether the minimum and maximum values are outside the bounds of 0, and max is 3.

```
range(timss[,Q15R])
```

```
## [1] 0 3
```

We could also look at each of the 7 questions:

```
apply(timss[, Q15R], 2, FUN = range)
```

```
##      Q15_StCentrd_aR Q15_StCentrd_bR Q15_StCentrd_cR Q15_StCentrd_dR
## [1,]              0              1              0              0
## [2,]              3              3              3              3
##      Q15_StCentrd_eR Q15_StCentrd_fR Q15_StCentrd_gR
## [1,]              0              0              0
## [2,]              3              3              3
```

None of the values is lower than 0 or higher than 3, which is what we expected.

2.3.1.2 Rescaling Q16 so zero is meaningful Question 16 has the numeric codes in a logical direction (where a smaller numeric value represents a lower amount on the concept), but let's change the scale from a 1-to-3 scale to 0-to-2 so that "Not at all" is zero.⁴ For this, we have to subtract 1 from the original value.

Let's remember what these Q16 variable names are:⁵

```
dput(names(timss))
```

```
## c("TID", "Q4_Age", "Q5_Degree", "Q6_Math", "Q6_Bio", "Q6_Phys",
## "Q6_Chem", "Q6_EarthS", "Q6_EdMat", "Q6_EdSci", "Q6_EdGen", "Q6_Othr",
## "Q13_NumSs", "Q15_StCentrd_a", "Q15_StCentrd_b", "Q15_StCentrd_c",
## "Q15_StCentrd_d", "Q15_StCentrd_e", "Q15_StCentrd_f", "Q15_StCentrd_g",
## "Q16_Lims_a", "Q16_Lims_b", "Q16_Lims_c", "Q16_Lims_d", "Q16_Lims_e",
## "Q16_Lims_f", "Q16_Lims_g", "Q17_Mins", "Q19_Didac_a", "Q19_Didac_b",
## "Q19_Didac_c", "Q6_Major_in_Edu", "Q15_StCentrd_aR", "Q15_StCentrd_bR",
## "Q15_StCentrd_cR", "Q15_StCentrd_dR", "Q15_StCentrd_eR", "Q15_StCentrd_fR",
## "Q15_StCentrd_gR")
```

Let's assign them to the object Q16:

```
Q16 <- c("Q16_Lims_a", "Q16_Lims_b", "Q16_Lims_c", "Q16_Lims_d", "Q16_Lims_e",
        "Q16_Lims_f", "Q16_Lims_g")
range(timss[, Q16])
```

```
## [1] 1 3
```

Because Q16 only needs to be lowered so that *Not at all* = 0, we might use "Z" in the names to remind us they are rescaled to have a meaningful zero.

```
Q16Z <- paste0(Q16, "Z")
Q16Z
```

```
## [1] "Q16_Lims_aZ" "Q16_Lims_bZ" "Q16_Lims_cZ" "Q16_Lims_dZ" "Q16_Lims_eZ"
## [6] "Q16_Lims_fZ" "Q16_Lims_gZ"
```

Now, let's subtract 1 from the original response and create our new variables in the data frame:

```
timss[, Q16Z] <- (timss[, Q16] - 1)
range(timss[, Q16Z])
```

```
## [1] 0 2
```

We can see these new variables are appended at the end of the data set:

```
str(timss)
```

⁴This recoding is helpful if we use these as predictors in a regression because we can interpret the intercept as a meaningful number.

⁵The `dput()` function creates an output that is easy to copy and paste into the `c()` function in our next step.

```
## 'data.frame':    30 obs. of  46 variables:
## $ TID           : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Q4_Age        : num  2 3 3 2 5 6 5 2 4 3 ...
## $ Q5_Degree     : num  4 4 4 5 3 4 5 4 4 4 ...
## $ Q6_Math       : chr  "No" "No" "No" "No" ...
## $ Q6_Bio        : chr  "No" "No" "No" "No" ...
## $ Q6_Phys       : chr  "No" "No" "Yes" "No" ...
## $ Q6_Chem       : chr  "No" "No" "No" "No" ...
## $ Q6_EarthS     : chr  "No" "No" "No" "No" ...
## $ Q6_EdMat      : chr  "No" "No" "No" "Yes" ...
## $ Q6_EdSci      : chr  "No" "No" "No" "No" ...
## $ Q6_EdGen      : chr  "Yes" "Yes" "No" "No" ...
## $ Q6_Othr       : chr  "No" "No" "Yes" "No" ...
## $ Q13_NumSs     : int  8 12 19 16 18 26 29 16 30 22 ...
## $ Q15_StCentrd_a : int  2 3 3 1 2 3 4 1 4 3 ...
## $ Q15_StCentrd_b : int  2 1 2 1 1 3 2 1 2 2 ...
## $ Q15_StCentrd_c : int  1 4 2 2 3 3 2 1 2 2 ...
## $ Q15_StCentrd_d : int  1 2 3 2 2 3 3 1 4 4 ...
## $ Q15_StCentrd_e : int  2 2 3 1 2 3 4 2 2 2 ...
## $ Q15_StCentrd_f : int  3 3 3 1 3 4 2 1 1 4 ...
## $ Q15_StCentrd_g : int  1 2 3 1 2 3 3 1 3 3 ...
## $ Q16_Lims_a     : int  2 1 2 1 1 2 3 1 3 1 ...
## $ Q16_Lims_b     : int  1 1 2 2 1 1 2 1 3 2 ...
## $ Q16_Lims_c     : int  3 3 3 3 3 3 3 2 2 2 ...
## $ Q16_Lims_d     : int  2 2 2 2 2 2 2 1 2 3 ...
## $ Q16_Lims_e     : int  1 1 2 2 1 2 2 2 2 2 ...
## $ Q16_Lims_f     : int  1 1 1 1 1 1 1 1 2 1 ...
## $ Q16_Lims_g     : int  2 1 2 1 1 1 1 2 1 1 ...
## $ Q17_Mins       : int  160 180 190 200 210 180 200 240 150 160 ...
## $ Q19_Didac_a    : int  3 4 2 3 3 3 2 3 2 1 ...
## $ Q19_Didac_b    : int  3 2 1 4 1 4 2 2 2 2 ...
## $ Q19_Didac_c    : int  4 3 2 3 2 2 2 1 2 1 ...
## $ Q6_Major_in_Edu: num  1 1 0 1 1 1 0 1 0 0 ...
## $ Q15_StCentrd_aR: num  2 1 1 3 2 1 0 3 0 1 ...
## $ Q15_StCentrd_bR: num  2 3 2 3 3 1 2 3 2 2 ...
## $ Q15_StCentrd_cR: num  3 0 2 2 1 1 2 3 2 2 ...
## $ Q15_StCentrd_dR: num  3 2 1 2 2 1 1 3 0 0 ...
## $ Q15_StCentrd_eR: num  2 2 1 3 2 1 0 2 2 2 ...
## $ Q15_StCentrd_fR: num  1 1 1 3 1 0 2 3 3 0 ...
## $ Q15_StCentrd_gR: num  3 2 1 3 2 1 1 3 1 1 ...
## $ Q16_Lims_aZ     : num  1 0 1 0 0 1 2 0 2 0 ...
## $ Q16_Lims_bZ     : num  0 0 1 1 0 0 1 0 2 1 ...
## $ Q16_Lims_cZ     : num  2 2 2 2 2 2 2 1 1 1 ...
## $ Q16_Lims_dZ     : num  1 1 1 1 1 1 1 0 1 2 ...
## $ Q16_Lims_eZ     : num  0 0 1 1 0 1 1 1 1 1 ...
## $ Q16_Lims_fZ     : num  0 0 0 0 0 0 0 0 1 0 ...
## $ Q16_Lims_gZ     : num  1 0 1 0 0 0 0 1 0 0 ...
```

Let's check that it worked with the first and 7th variable.⁶

⁶We could also copy and paste the code and replace the numbers with any within the range of 1 through 7 because there are 7 items in this question.

```
timss[7:12, c(Q16[1], Q16Z[1])]
timss[7:12, c(Q16[7], Q16Z[7])]
```

2.3.1.3 Rescaling Q19 Similar to Question 15, Question 19’s response scale categories are in an illogical direction. Let’s do the same thing we did for Question 15.

```
str(timss)

Q19 <- c("Q19_Didac_a", "Q19_Didac_b", "Q19_Didac_c")
Q19R<- c("Q19_Didac_aR", "Q19_Didac_bR", "Q19_Didac_cR")

timss[,Q19R] <- (4 + 0 - timss[,Q19])
```

2.4 Creating Composite Scores (AKA Indexes)

1) Let’s create an index (AKA scale or composite) variable from the 7 numeric items in Q15. Let’s name our new variable Q15_Stdnt_Centred_Index.

```
timss$Q15_Index <- apply(timss[,Q15R],1, mean, na.rm = TRUE)
```

Applying the mean function across columns “1” directs R to apply this function to the first margin, i.e. to the rows (and across columns). See http://www2.hawaii.edu/~georgeha/surveys/_book/appending.html. So we get each person’s mean score across the 7 items in Q15R. `na.rm = TRUE` is used in case we have any missing data.

Let’s look at the index scores for the first 5 or so observations:

```
timss[1:5, c(Q15R,"Q15_Index")]
```

```
##   Q15_StCentrd_aR Q15_StCentrd_bR Q15_StCentrd_cR Q15_StCentrd_dR
## 1                2                2                3                3
## 2                1                3                0                2
## 3                1                2                2                1
## 4                3                3                2                2
## 5                2                3                1                2
##   Q15_StCentrd_eR Q15_StCentrd_fR Q15_StCentrd_gR Q15_Index
## 1                2                1                3  2.285714
## 2                2                1                2  1.571429
## 3                1                1                1  1.285714
## 4                3                3                3  2.714286
## 5                2                1                2  1.857143
```

```
summary(timss$Q15_Index)
```

```
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.5714  1.2857  1.5714  1.7190  2.0000  3.0000
```

We can use the structure function, `str()`, with the same subset of data:

```
str(timss[, c(Q15R,"Q15_Index")])
```

```
## 'data.frame': 30 obs. of 8 variables:
## $ Q15_StCentrd_aR: num 2 1 1 3 2 1 0 3 0 1 ...
## $ Q15_StCentrd_bR: num 2 3 2 3 3 1 2 3 2 2 ...
## $ Q15_StCentrd_cR: num 3 0 2 2 1 1 2 3 2 2 ...
## $ Q15_StCentrd_dR: num 3 2 1 2 2 1 1 3 0 0 ...
## $ Q15_StCentrd_eR: num 2 2 1 3 2 1 0 2 2 2 ...
## $ Q15_StCentrd_fR: num 1 1 1 3 1 0 2 3 3 0 ...
## $ Q15_StCentrd_gR: num 3 2 1 3 2 1 1 3 1 1 ...
## $ Q15_Index : num 2.29 1.57 1.29 2.71 1.86 ...
```

2) Let's do the same thing with Q16 and Q19:

```
timss$Q19_Index <- apply(timss[,Q19R], 1, mean, na.rm = TRUE)
timss$Q16_Index <- apply(timss[,Q16Z], 1, mean, na.rm = TRUE)
```

Take a look at the new composite variables:

```
timss[1:5,c(Q19R,"Q19_Index")]
```

```
## Q19_Didac_aR Q19_Didac_bR Q19_Didac_cR Q19_Index
## 1 1 1 0 0.6666667
## 2 0 2 1 1.0000000
## 3 2 3 2 2.3333333
## 4 1 0 1 0.6666667
## 5 1 3 2 2.0000000
```

```
timss[1:5,c(Q16Z,"Q16_Index")]
```

```
## Q16_Lims_aZ Q16_Lims_bZ Q16_Lims_cZ Q16_Lims_dZ Q16_Lims_eZ Q16_Lims_fZ
## 1 1 0 2 1 0 0
## 2 0 0 2 1 0 0
## 3 1 1 2 1 1 0
## 4 0 1 2 1 1 0
## 5 0 0 2 1 0 0
## Q16_Lims_gZ Q16_Index
## 1 1 0.7142857
## 2 0 0.4285714
## 3 1 1.0000000
## 4 0 0.7142857
## 5 0 0.4285714
```

2.5 A general formula for rescaling data

Here's a more general formula for rescaling:

Sometimes, we want to rescale data. For example, if we want to create an index based on several variables, but some of these variables are on a 1-to-5 scale while others are on a 1-to-6 scale and we want to give equal weight to the variables, we would rescale one type to be on the other's scale.

We can use this general formula, with the “bottom” and the “top” referring to the bottom and top of the scale and the “value” referring to the response of interest (a person’s response): $\text{new.value} = \text{new.bottom} + (\text{old.value} - \text{old.bottom}) * (\text{new.top} - \text{new.bottom}) / (\text{old.top} - \text{old.bottom})$ but first we have to assign values to each of the objects on the right side (or we could just plug them in). Here is an example of rescaling a response of 3 that was on a scale of 1 to 6 to the new scale of 1 to 5:

```
old.value <- 3 #This is our original data point.
old.bottom<- 1 #This is the lowest possible point on the original scale
old.top    <- 6 #This is the highest possible point on the original scale
new.bottom<- 1 #This is the lowest possible point on the new scale
new.top    <- 5 #This is the highest possible point on the new scale

new.bottom + (old.value - old.bottom)*(new.top - new.bottom)/(old.top - old.bottom)

## [1] 2.6
```

which is the same as this if we weren’t using object names:

```
1 + (3 - 1)*(5-1)/(6-1)

## [1] 2.6
```

Seems like too much work, right. However, we can use this in a custom function.

2.6 Create and use a custom function

It’s convenient to create a custom function using that formula. Let’s name our custom function something that does not already exist in base R. We can check this using the `help()` function to see if something comes up:

```
help("myrescale")
```

Seems like “myrescale” is a good name because nothing comes up in help. After the `function()`, we specify the argument names and their order. Then, we enclose the operations in the curly brackets. The final object, `new.value`, is what is produced.

```
myrescale <- function(old.value, old.bottom, old.top, new.bottom, new.top){
  new.value <- new.bottom + (old.value - old.bottom)*(new.top - new.bottom)/(old.top - old.bottom)
}
```

Now we can use that custom function. Let’s make up some arbitrary original data, within our original 1 to 6 scale:

```
olddat <- c(4, 3, 2, 1, 6, 5)

new1to5 <- myrescale(old.value = olddat,
                     old.bottom = 1,
                     old.top    = 6,
                     new.bottom = 1,
                     new.top    = 5)

new1to5
```

```
## [1] 3.4 2.6 1.8 1.0 5.0 4.2
```

So, we used our homemade function, `myrescale()`, by specifying the arguments. We can re-use this function in our R session.

Here's another example with our data set: Maybe for some reason we want to transform all of our Q15R variables to a 5 point scale that is centered at the midpoint, so from -2 to 2:

First let's create a name for our new set of variables Maybe we use "C5" to remind ourselves it's centered and on five points:

```
Q15Centr5 <- paste0(Q15, "_C5")
Q15Centr5
```

```
## [1] "Q15_StCentrd_a_C5" "Q15_StCentrd_b_C5" "Q15_StCentrd_c_C5"
## [4] "Q15_StCentrd_d_C5" "Q15_StCentrd_e_C5" "Q15_StCentrd_f_C5"
## [7] "Q15_StCentrd_g_C5"
```

```
timss[,Q15Centr5]<- myrescale(old.value = timss[, Q15R],
                             old.bottom = 0,
                             old.top    = 3,
                             new.bottom = -2,
                             new.top    = 2)
timss[1:6, c(Q15R[3], Q15Centr5[3]) ]
```

```
##   Q15_StCentrd_cR Q15_StCentrd_c_C5
## 1                3      2.0000000
## 2                0     -2.0000000
## 3                2      0.6666667
## 4                2      0.6666667
## 5                1     -0.6666667
## 6                1     -0.6666667
```

We can use this general formula to reverse scale, too. To reverse the scale, the new top and bottom are just reversed:

```
newReversed <- myrescale(olddat,
                        old.bottom = 1,
                        old.top    = 6,
                        new.bottom = 6,
                        new.top    = 1)
```

Let's compare our newly reversed scale to our original scale:

```
newReversed
```

```
## [1] 3 4 5 6 1 2
```

```
olddat
```

```
## [1] 4 3 2 1 6 5
```

2.7 The code in one place for one of the composite scores

Here is the code again for one of the scales—the Q15 index—but without the extra steps above. This shows the core steps:

1. Reading in the CSV file,
2. Examining the structure of the data frame,
3. Assigning the names of the columns to an object, Q15, so we can use that object in subsequent code,
4. Creating a new object, Q15R that includes the names of the Q15 items but with an “R” so we can create a new set of columns that are rescaled,
5. The actual rescaling of the items,
6. Calculating the composite score from the rescaled items, and
7. Examining the structure of our rescaled items and our composite score.

```
timss <- read.csv('Week13_Timss_Mock_Raw.csv')
str(timss)
Q15 <- c("Q15_StCentrd_a", "Q15_StCentrd_b", "Q15_StCentrd_c", "Q15_StCentrd_d",
        "Q15_StCentrd_e", "Q15_StCentrd_f", "Q15_StCentrd_g")
Q15R <- paste0(Q15,"R")
timss[,Q15R] <- (4 + 0 - timss[,Q15])
timss$Q15_Index <- apply(timss[,Q15R],1, mean, na.rm = TRUE)
str(timss[, c(Q15R,"Q15_Index")])
```

2.8 A shortcut, when all of the items are on the same scale

So far, we have been practicing rescaling items before we use them to create a composite score. That was worth learning because sometimes we find ourselves needing to recode only some of the items (e.g., if we need to reverse-code items that are phrased in the opposite direction to the other items in the scale).

When we do not have that situation, we can take a simpler approach: We can create the composite variable and then rescale that. Here’s an example with Q15.

```
Q15 <- paste0("Q15_StCentrd_", letters[1:7])
timss$Q15_Index_raw <- apply(timss[,Q15],1, mean, na.rm = TRUE)
timss$Q15_Index_easy <- 4 + 0 - timss$Q15_Index_raw
```

Annoying, huh. We did all of the work above when we could have achieved the same result with three lines of code. Just to verify, in this demo, let’s eyeball the first 10 people’s scores on this variable (timss\$Q15_Index_easy) and the one we had created earlier using after rescaling the individual items (Q15_Index):

```
timss[1:10, c("TID", "Q15_Index", "Q15_Index_easy")]
```

##	TID	Q15_Index	Q15_Index_easy
## 1	1	2.2857143	2.2857143
## 2	2	1.5714286	1.5714286
## 3	3	1.2857143	1.2857143
## 4	4	2.7142857	2.7142857
## 5	5	1.8571429	1.8571429
## 6	6	0.8571429	0.8571429
## 7	7	1.1428571	1.1428571
## 8	8	2.8571429	2.8571429
## 9	9	1.4285714	1.4285714
## 10	10	1.1428571	1.1428571

2.9 Summary

In this chapter, we created variables (dummy codes and composite scores), recoded and rescaled variables (items and composite variables), created a custom function, and practiced looking at our data and the structure of our data.

3 Installing and Using Packages

In R, we use packages that are developed by nerdy people (whom I admire) and made available for the rest of us to use. Often a package is for a specific purpose or a family of analyses. To analyze simple survey data, we will use the `psych` package [Revelle, 2020] to perform descriptive statistics and estimate reliability coefficients, the `ggplot2` package [Wickham et al., 2020a] to create pretty plots, and the `dplyr` [Wickham et al., 2020b] package to wrangle our data.

3.1 Seeing if we need to install a package

First, we need to see if the packages we need for this session are installed on our machine. To do this, we can either

1. see if each of these packages is listed in the “Packages” tab in the lower right pane, or
2. use the `require()` function with each package to see if we get the result “there is no package called ‘[package name]’”. Let’s say we think there’s a package called `PackageNot` on our machine. (You can plug in any random name for this.) We can verify this with this code:

```
require("PackageNot")
```

```
## Loading required package: PackageNot
```

```
## Warning in library(package, lib.loc = lib.loc, character.only = TRUE,  
## logical.return = TRUE, : there is no package called 'PackageNot'
```

Do the same thing with the `psych` package. If you do *not* have it installed, it will give you the same result. If you do have it installed, it will generate an output that says `Loading required package: psych`:

```
require("psych")
```

```
## Loading required package: psych
```

You might also see a warning that the package was built under a newer version of R. There are R updates every few months or so. If you find the package is not working, you might need to update R, which we won’t address right now.

3.2 Installing a package

The `install.packages()` function is handy for installing packages:

```
install.packages("psych")
```

Once a package is installed on a machine, we do not need to install it again.⁷ But, you might want to keep your code as a record of what you did or make the `install.packages()` line of your code accessible to your friends if you wish to share it with others.⁸ You can comment out the code if you want R to ignore it. In R, anything after a `#` symbol is commented out:

```
# install.packages("psych")
```

Now, let's bring the package into our working library so we can use a function from that package.⁹

```
library(psych)
```

We can also find out what functions are available in a package with the `help()` function and the package name:

```
help(package = "psych")
```

The functions are in alphabetical order. We can see the `alpha()` function listed second in the alphabet (under `-- A --`). We will use this function for estimating internal consistency reliability. If you click on the function, there will be a lot of information to sort through. The **Usage** and **Arguments** sections near the top and the **Examples** section at the bottom are usually useful.

3.3 Using a function from a package

After we bring the package into our library, during our R session, we can use its functions.

One that we will use is the `describe()` function. First, let's review some code that we learned last week:¹⁰

```
timss <- read.csv('Week13_Timss_Mock_Raw.csv')

Q15 <- c("Q15_StCentrd_a", "Q15_StCentrd_b", "Q15_StCentrd_c", "Q15_StCentrd_d",
        "Q15_StCentrd_e", "Q15_StCentrd_f", "Q15_StCentrd_g")

Q15R <- paste0(Q15,"R")

timss[,Q15R] <- (4 + 0 - timss[,Q15])

timss$Q15_Index <- apply(timss[,Q15R],1, mean, na.rm = TRUE)
```

We could use the `summary()` function from Base R to get some descriptive statistics of this variable, `Q15_Index`.

⁷It is a good idea, however, to update the packages every once in a while. In RStudio, we can click on the Packages tab and click Update.

⁸Talking about R functions in social situations is a great way to ensure you get more alone time.

⁹Base R has a lot of inconsistencies. You may notice that we need quotes around the package name when we use the `install.packages()` function but we don't when we use the `library()` function. This is part of the so-called fun in R.

¹⁰Also, note that because all of our items require the same rescaling procedure, we could use a shorter version of this code, discussed at the end of the last chapter.

```
summary(timss$Q15_Index)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.5714  1.2857  1.5714  1.7190  2.0000  3.0000
```

This does not show the standard deviation or the n -size. Now, let's try the `describe()` function from the `psych` package:

```
describe(timss$Q15_Index)
```

```
##      vars  n mean   sd median trimmed  mad  min max range skew kurtosis   se
## X1      1 30 1.72 0.61   1.57   1.68 0.64 0.57   3  2.43 0.43   -0.62 0.11
```

With this, we get the n -size (`n`), the standard deviation (`sd`), and some other handy estimates, such as the standard error of the mean (`se` in this output). We might not wish to report all of these numbers from the output, but this is certainly more informative than Base R's `summary()` function.

4 Estimating Descriptive Statistics

With surveys, we typically report the descriptive statistics of the individual items and of the scales—the composite variables—that we calculated from those items. There are many ways in R to go about estimating the descriptive statistics.¹¹ An easy way is to use the `describe()` function from the `psych` package. We will play with that. Another way is to use Base R's functions and create our own function.¹² This is handy, particularly if a descriptive statistic is not available to us in a package. We will use this approach, too, because we need to report the confidence intervals of the mean estimate—which is not something that seems to be available with the `psych` package's `describe()` function.

We also need to remember that there are different levels of data. For categorical data, such as those on nominal and ordinal level scales, it is a good idea to report the frequency (and percentages) of responses in each category. For interval level data, we can report the means, standard deviations, standard error of the mean, and so forth.

4.1 Descriptive statistics of categorical data

After we've imported the data, we can examine the frequency counts of the categorical variables. With our data, we can see some variables are numeric (`int` or `num`) and some are character (`chr`) in type:

```
timss <- read.csv('Week13_Timss_Mock_Raw.csv')
str(timss)
```

```
## 'data.frame':   30 obs. of  31 variables:
##  $ TID          : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ Q4_Age       : num  2 3 3 2 5 6 5 2 4 3 ...
##  $ Q5_Degree    : num  4 4 4 5 3 4 5 4 4 4 ...
##  $ Q6_Math      : chr  "No" "No" "No" "No" ...
```

¹¹Notice that I said “estimate descriptive statistics.” There is always some degree of uncertainty about how closely our estimates reflect the true status in the population.

¹²The term *Base R* is used to describe the functions that come with R rather than those that are part of packages that we install and use.

```
## $ Q6_Bio      : chr "No" "No" "No" "No" ...
## $ Q6_Phys     : chr "No" "No" "Yes" "No" ...
## $ Q6_Chem     : chr "No" "No" "No" "No" ...
## $ Q6_EarthS   : chr "No" "No" "No" "No" ...
## $ Q6_EdMat    : chr "No" "No" "No" "Yes" ...
## $ Q6_EdSci    : chr "No" "No" "No" "No" ...
## $ Q6_EdGen    : chr "Yes" "Yes" "No" "No" ...
## $ Q6_Othr     : chr "No" "No" "Yes" "No" ...
## $ Q13_NumSs   : int  8 12 19 16 18 26 29 16 30 22 ...
## $ Q15_StCentrd_a: int  2 3 3 1 2 3 4 1 4 3 ...
## $ Q15_StCentrd_b: int  2 1 2 1 1 3 2 1 2 2 ...
## $ Q15_StCentrd_c: int  1 4 2 2 3 3 2 1 2 2 ...
## $ Q15_StCentrd_d: int  1 2 3 2 2 3 3 1 4 4 ...
## $ Q15_StCentrd_e: int  2 2 3 1 2 3 4 2 2 2 ...
## $ Q15_StCentrd_f: int  3 3 3 1 3 4 2 1 1 4 ...
## $ Q15_StCentrd_g: int  1 2 3 1 2 3 3 1 3 3 ...
## $ Q16_Lims_a    : int  2 1 2 1 1 2 3 1 3 1 ...
## $ Q16_Lims_b    : int  1 1 2 2 1 1 2 1 3 2 ...
## $ Q16_Lims_c    : int  3 3 3 3 3 3 3 2 2 2 ...
## $ Q16_Lims_d    : int  2 2 2 2 2 2 2 1 2 3 ...
## $ Q16_Lims_e    : int  1 1 2 2 1 2 2 2 2 2 ...
## $ Q16_Lims_f    : int  1 1 1 1 1 1 1 1 2 1 ...
## $ Q16_Lims_g    : int  2 1 2 1 1 1 1 2 1 1 ...
## $ Q17_Mins      : int 160 180 190 200 210 180 200 240 150 160 ...
## $ Q19_Didac_a   : int  3 4 2 3 3 3 2 3 2 1 ...
## $ Q19_Didac_b   : int  3 2 1 4 1 4 2 2 2 2 ...
## $ Q19_Didac_c   : int  4 3 2 3 2 2 2 1 2 1 ...
```

One of the variables in our data set is Q5_Degree, but it is coded as having numeric values. This is an ordinal level variable. The numbers in the data correspond to these categories on the survey instrument:

- 1 = *Did not complete high school*
- 2 = *High school graduate*
- 3 = *Associate's degree*
- 4 = *Bachelor's degree*
- 5 = *Master's degree*
- 6 = *Doctorate*

To get the frequency counts of the responses in our data, we can use the `table()` function on the item:

```
table(timss$Q5_Degree)
```

```
##
##  3  4  5  6
##  1 20  8  1
```

So, there was 1 person whose highest degree was at the associate's level, 20 with a bachelor's degree as their highest, 8 with at least a master's degree, and 1 with a doctorate.

We can calculate the proportion (or percentage) of the responses that are in each category using the `prop.table()` function around the `table` function.

```
prop.table(table(timss$Q5_Degree))
```

```
##
##           3           4           5           6
## 0.03333333 0.66666667 0.26666667 0.03333333
```

We can see that 2/3 of the teachers (0.67) have a bachelor's as their highest degree.

4.1.1 Creating factors

Instead of having R treat these as numeric data, which would incorrectly permit calculation of the means and other types of operations that assume an interval level scale, we can change the variable type to be an *ordered factor*. We can use the `factor()` function.¹³

```
timss$Q5_Degree_F <- factor(timss$Q5_Degree,
                             ordered = TRUE,
                             levels = c("1",
                                          "2",
                                          "3",
                                          "4",
                                          "5",
                                          "6"),
                             labels = c("No_HS",
                                          "HS",
                                          "Associates",
                                          "Bachelors",
                                          "Masters",
                                          "Doctorate"))
str(timss$Q5_Degree_F)
```

```
## Ord.factor w/ 6 levels "No_HS"<"HS"<"Associates"<...: 4 4 4 5 3 4 5 4 4 4 ...
```

Now that we have labeled the levels in the factor, the output from the `table()` and `prop.table()` functions is easier to interpret.¹⁴ Factors have a specific set of categories (levels), so when we ask for the frequency of responses in a factor level and zero observations are present, it will still report it (as 0).

```
table(timss$Q5_Degree_F)
```

```
##
##      No_HS      HS Associates Bachelors  Masters  Doctorate
##         0         0          1         20         8          1
```

```
round( prop.table(table(timss$Q5_Degree_F)), 2)
```

```
##
##      No_HS      HS Associates Bachelors  Masters  Doctorate
##      0.00      0.00      0.03      0.67      0.27      0.03
```

¹³This term *factor* comes from the ANOVA framework, in which a factor has levels. This is *not* the same thing as a factor in factor analysis, which corresponds to a dimension or latent construct in the data.

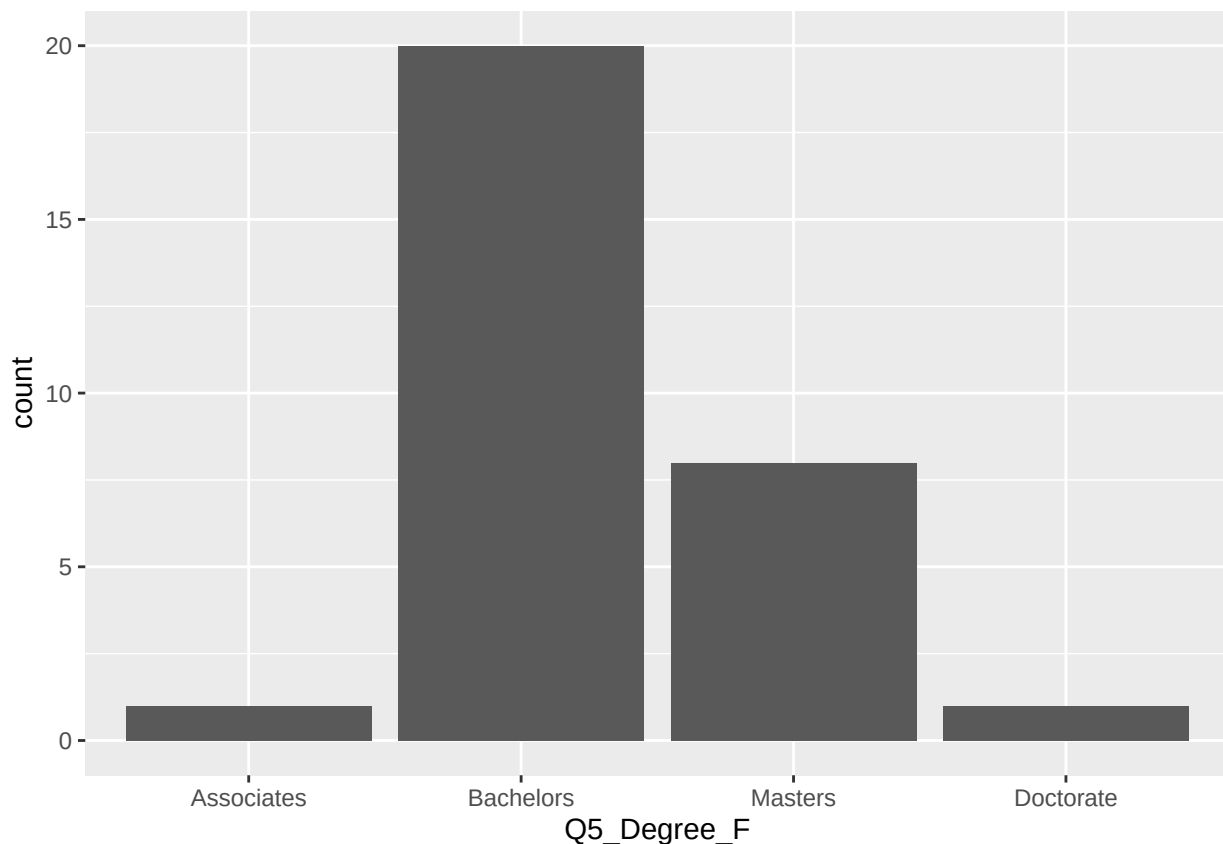
¹⁴Here, we're using the `round()` function to make the output even easier to read.

Specifying our variables as factors is not always required for simple analyses. It's a good idea to have two versions, one that is a numeric, dummy-coded, version that we can include in a regression and another that is a factor, which shows up nicely in plots.

4.1.2 Plotting the frequency data

Let's use the `ggplot2` package to plot the data. First be sure to install the package, which you can do by running `install.packages("ggplot2")`.

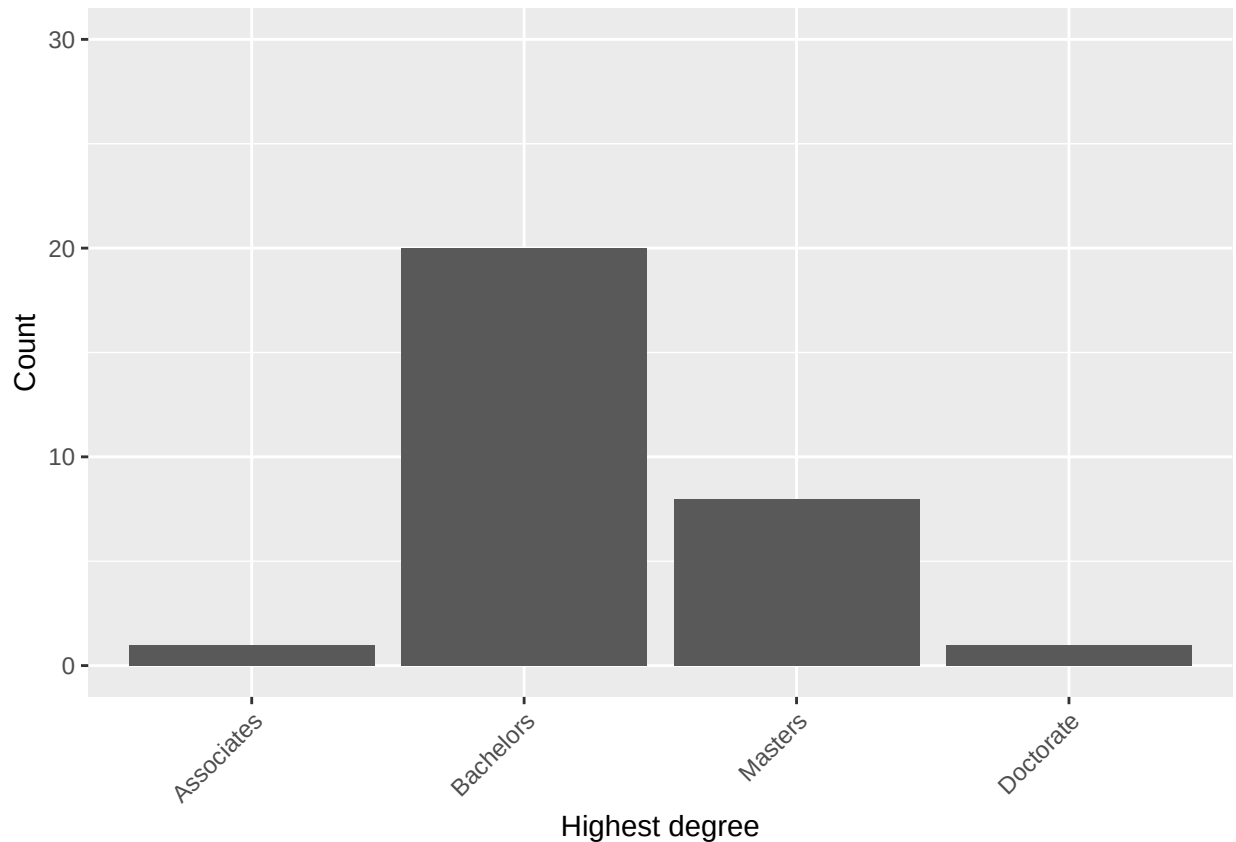
```
library(ggplot2)
ggplot(data = timss, aes(x = Q5_Degree_F) ) +
  geom_bar()
```



The `ggplot()` function includes the data frame in the `data =` argument, which in this case is `timss`. Then, there's the `aes()` argument, which refers to the aesthetics. This is where we would identify the X and Y axes. Here, we're only using an X axis. That first line creates the plotting area but does not create the plot yet because we have to specify which type of plot, which is what the `geom_bar()` in the second line does. Notice that after each line in `ggplot`, we add a `+`. This tells `ggplot` to add the next line to the plot.

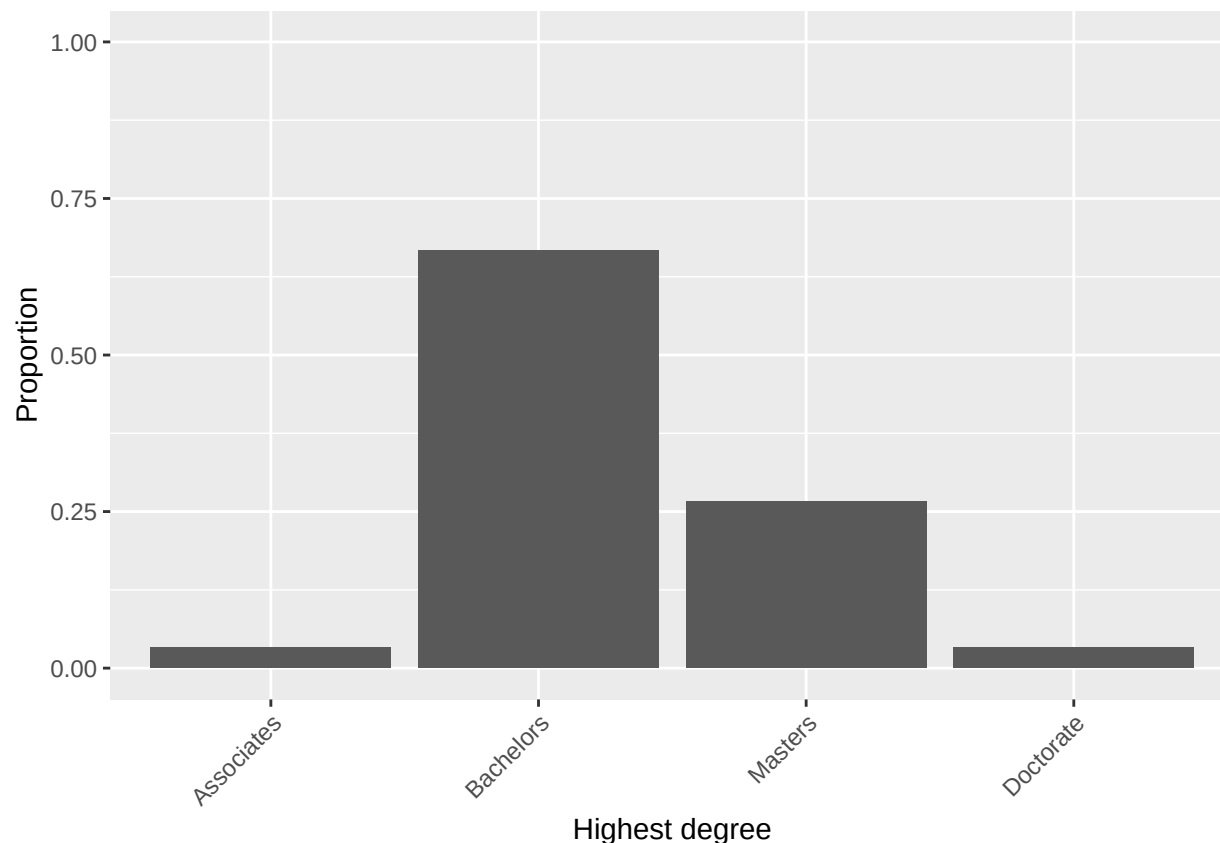
We can make modify the plot. One thing we might do is place the scale of the Y axis from 0 to the total number of observations (`nrow(timss)`), which is 30 in our data. For this, we can use `ylim(0, 30)` or `ylim(0, nrow(timss))`. We can change the labels of the x and y axis and change the angle of the text on the x axis.

```
ggplot(data = timss, aes(x = Q5_Degree_F)) +
  geom_bar() +
  ylim(0, nrow(timss)) +
  xlab("Highest degree" ) + ylab("Count") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



We can plot the proportions on the Y axis, too, if we think that is important. Notice that we can include an `aes()` argument in the `geom_bar()` line. I found the `(..count..)/sum(..count..)` part of the code in an online post rather than recalling it from memory but this is an example of how we can modify the y axis.

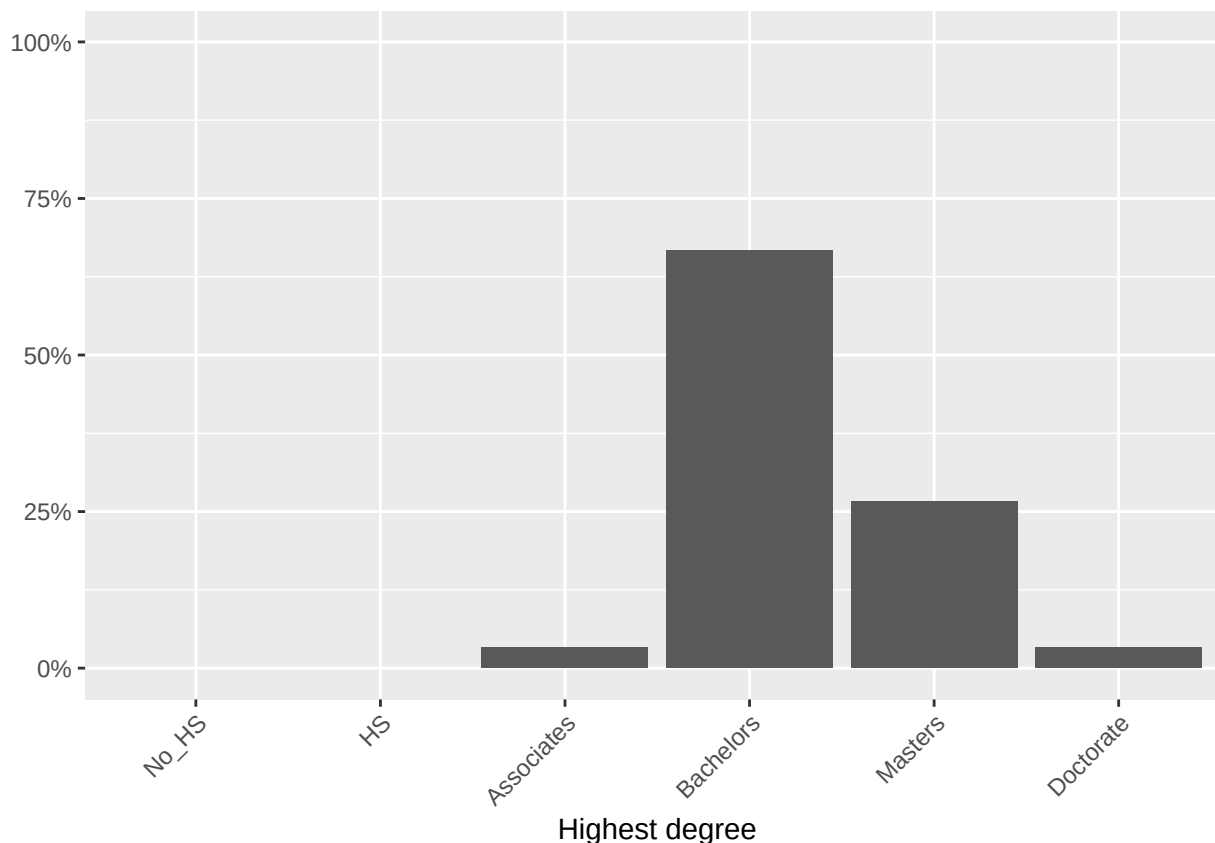
```
ggplot(data = timss, aes(x = Q5_Degree_F)) +
  geom_bar(aes(y = (..count..)/sum(..count..))) +
  ylim(0, 1) +
  xlab("Highest degree" ) + ylab("Proportion") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



There are many ways to accomplish the same thing in ggplot. It is powerful but the details are hard to remember.¹⁵ Here, we can change the Y axis to be percentages. If we wanted to include “Percentage” on the Y axis label, we would change `name = NULL` to `name = "Percentage"` in the code below:

```
ggplot(data = timss, aes(x = Q5_Degree_F)) +
  geom_bar(aes(y = (..count..)/sum(..count..))) +
  xlab("Highest degree" ) +
  scale_y_continuous(labels = scales::percent, limits = c(0,1), name = NULL) + # This line formats the Y
  scale_x_discrete(drop = FALSE) + # You can keep this line if you want to include categories that have
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

¹⁵If you remember the main function, the aesthetics argument and the geoms, you’re in good shape because we often find examples rather quickly in online searches.



4.1.3 Cross-tabulations

We can also create cross-tabulations—also called contingency tables—of categorical data. For instance, in the first chapter, we created a variable that identified whether the respondent had majored in education or not. We can see what the education level breakdowns are within each of these two categories:

```
timss$Q6_Major_in_Edu <- ifelse(timss$Q6_EdMat == "Yes" |
                                timss$Q6_EdSci == "Yes" |
                                timss$Q6_EdGen == "Yes", 1, 0)
```

Optionally, we can create a factor version of this variable:

```
timss$Q6_Major_in_Edu_F <- factor(timss$Q6_Major_in_Edu,
                                  levels = c(0,1),
                                  labels = c("Not_education",
                                              "Education"))
```

The simplest function is to use the `table()` function again. We see two rows, one for each unique value in the first variable.¹⁶ We see the six categories of the second variable as the columns:

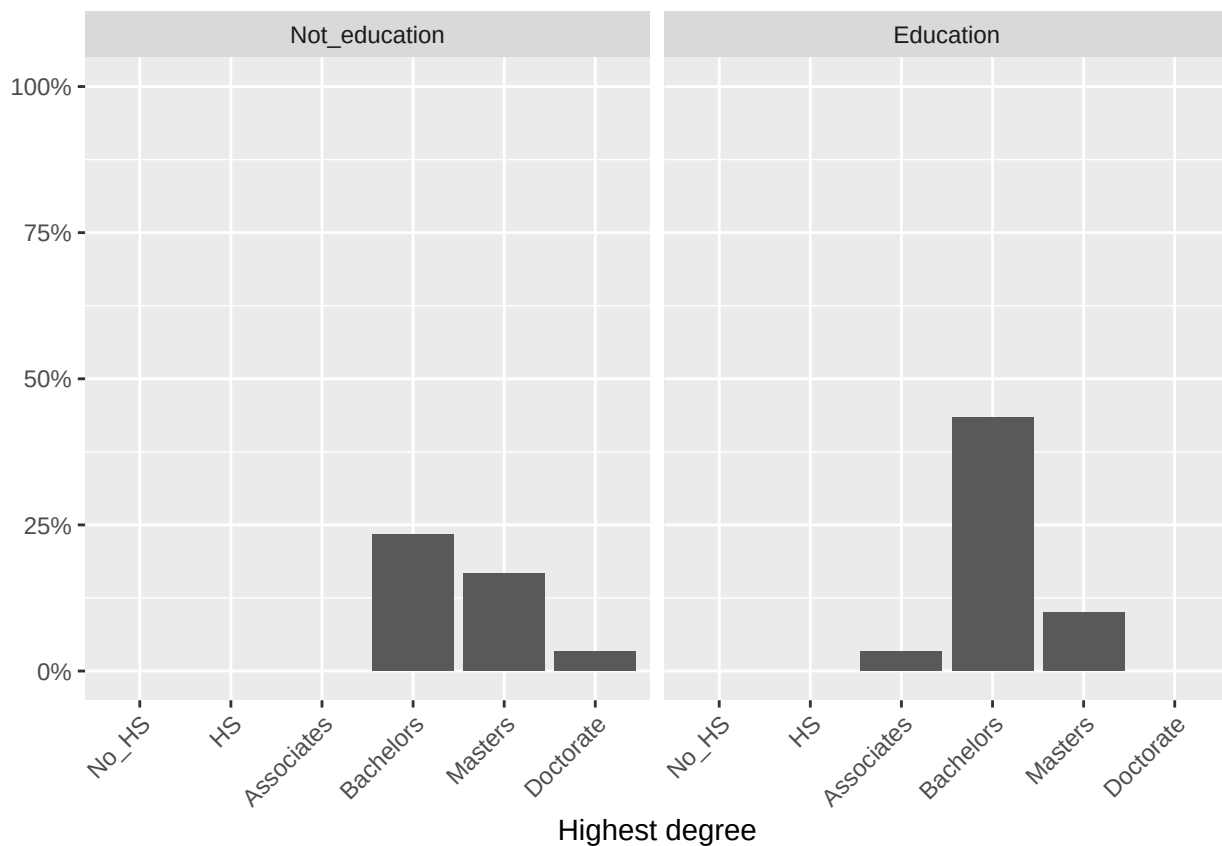
```
table(timss$Q6_Major_in_Edu_F, timss$Q5_Degree_F)
```

¹⁶We could have used the regular dummy variable with 0 and 1 values, too. The factor labeling makes it easier to interpret.

```
##
##           No_HS HS Associates Bachelors Masters Doctorate
## Not_education  0  0           0           7           5           1
## Education      0  0           1          13           3           0
```

We can create a facet plot to display these side by side. Here, we add a `facet_grid()` line with the categorical variable that we want the facets to be split up by. Don't worry if you cannot remember the details of all of this code—the purpose here is to understand what is possible. Often we search for this syntax when we need it.

```
ggplot(data = timss, aes(x = Q5_Degree_F)) +
  geom_bar(aes(y = (..count..)/sum(..count..))) +
  xlab("Highest degree" ) +
  scale_y_continuous(labels = scales::percent, limits = c(0,1), name = NULL) +
  scale_x_discrete(drop = FALSE) +
  facet_grid(~ Q6_Major_in_Edu_F) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



4.2 Descriptive statistics of interval-level data

For this part, let's use the composite variables we created in the first chapter. Here's the shortcut version of that code:¹⁷

¹⁷This shortcut code is calculating the composite scores on the raw scale and *then* rescaling that composite score, as demonstrated at the end of the last chapter. This works because all of the individual items required the same rescaling procedure. If some items in a composite scale require one type of rescaling, such as reverse coding, while others require other scaling, we would need to do this at the item level—which is the longer version presented in the earlier chapter.

```

# Reverse scoring the composite Q15 scale:
Q15 <- paste0("Q15_StCentrd_", letters[1:7])
timss$Q15_Index_raw <- apply(timss[,Q15],1, mean, na.rm = TRUE)
timss$Q15_Index <- 4 + 0 - timss$Q15_Index_raw

# Lower the composite Q16 scale by 1 point so zero has meaning
Q16 <- paste0("Q16_Lims_", letters[1:7])
timss$Q16_Index_raw <- apply(timss[,Q16],1, mean, na.rm = TRUE)
timss$Q16_Index <- timss$Q16_Index_raw - 1

# Reverse scoring the composite Q19 scale:
Q19 <- paste0("Q19_Didac_", letters[1:3])
timss$Q19_Index_raw <- apply(timss[,Q19],1, mean, na.rm = TRUE)
timss$Q19_Index <- 4 + 0 - timss$Q19_Index_raw

str(timss)

```

```

## 'data.frame':    30 obs. of  40 variables:
## $ TID           : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Q4_Age        : num  2 3 3 2 5 6 5 2 4 3 ...
## $ Q5_Degree      : num  4 4 4 5 3 4 5 4 4 4 ...
## $ Q6_Math        : chr  "No" "No" "No" "No" ...
## $ Q6_Bio         : chr  "No" "No" "No" "No" ...
## $ Q6_Phys        : chr  "No" "No" "Yes" "No" ...
## $ Q6_Chem        : chr  "No" "No" "No" "No" ...
## $ Q6_EarthS      : chr  "No" "No" "No" "No" ...
## $ Q6_EdMat       : chr  "No" "No" "No" "Yes" ...
## $ Q6_EdSci       : chr  "No" "No" "No" "No" ...
## $ Q6_EdGen       : chr  "Yes" "Yes" "No" "No" ...
## $ Q6_Othr        : chr  "No" "No" "Yes" "No" ...
## $ Q13_NumSs      : int  8 12 19 16 18 26 29 16 30 22 ...
## $ Q15_StCentrd_a : int  2 3 3 1 2 3 4 1 4 3 ...
## $ Q15_StCentrd_b : int  2 1 2 1 1 3 2 1 2 2 ...
## $ Q15_StCentrd_c : int  1 4 2 2 3 3 2 1 2 2 ...
## $ Q15_StCentrd_d : int  1 2 3 2 2 3 3 1 4 4 ...
## $ Q15_StCentrd_e : int  2 2 3 1 2 3 4 2 2 2 ...
## $ Q15_StCentrd_f : int  3 3 3 1 3 4 2 1 1 4 ...
## $ Q15_StCentrd_g : int  1 2 3 1 2 3 3 1 3 3 ...
## $ Q16_Lims_a     : int  2 1 2 1 1 2 3 1 3 1 ...
## $ Q16_Lims_b     : int  1 1 2 2 1 1 2 1 3 2 ...
## $ Q16_Lims_c     : int  3 3 3 3 3 3 3 2 2 2 ...
## $ Q16_Lims_d     : int  2 2 2 2 2 2 2 1 2 3 ...
## $ Q16_Lims_e     : int  1 1 2 2 1 2 2 2 2 2 ...
## $ Q16_Lims_f     : int  1 1 1 1 1 1 1 1 2 1 ...
## $ Q16_Lims_g     : int  2 1 2 1 1 1 1 2 1 1 ...
## $ Q17_Mins       : int  160 180 190 200 210 180 200 240 150 160 ...
## $ Q19_Didac_a    : int  3 4 2 3 3 3 2 3 2 1 ...
## $ Q19_Didac_b    : int  3 2 1 4 1 4 2 2 2 2 ...
## $ Q19_Didac_c    : int  4 3 2 3 2 2 2 1 2 1 ...
## $ Q5_Degree_F    : Ord.factor w/ 6 levels "No_HS"<"HS"<"Associates"<...: 4 4 4 5 3 4 5 4 4 4 ...
## $ Q6_Major_in_Edu : num  1 1 0 1 1 1 0 1 0 0 ...
## $ Q6_Major_in_Edu_F: Factor w/ 2 levels "Not_education",...: 2 2 1 2 2 2 1 2 1 1 ...
## $ Q15_Index_raw   : num  1.71 2.43 2.71 1.29 2.14 ...

```

```
## $ Q15_Index      : num  2.29 1.57 1.29 2.71 1.86 ...
## $ Q16_Index_raw  : num  1.71 1.43 2 1.71 1.43 ...
## $ Q16_Index      : num  0.714 0.429 1 0.714 0.429 ...
## $ Q19_Index_raw  : num  3.33 3 1.67 3.33 2 ...
## $ Q19_Index      : num  0.667 1 2.333 0.667 2 ...
```

While we are at this, we might as well estimate the descriptive statistics of two more variables, Q13_NumSs, and Q17_Mins, which are the number of students in the teacher's math class and the number of minutes in a typical week that the teacher spends teaching math to those students. So, we have five continuous variables we need to report on.

We could use the `summary()` function for each one, but this is pretty limiting. For example, it does not include standard deviation or the standard error of the mean.

```
summary(timss$Q15_Index)
```

```
##      Min. 1st Qu.  Median      Mean 3rd Qu.      Max.
## 0.5714  1.2857  1.5714  1.7190  2.0000  3.0000
```

We can use specific functions for the statistics that are missing from `summary()`, such as `sd()`:

```
sd(timss$Q15_Index, na.rm = TRUE)
```

```
## [1] 0.6054913
```

The `na.rm = TRUE` tells R to calculate the value even if there are missing data (i.e., remove an observation before the calculation). Most other programs, like SPSS do this by default, but R is making us think twice about this, which is actually a good thing.

We could also use our own function to calculate the standard error of the mean, $\frac{s_x}{\sqrt{n_x}}$.¹⁸

```
Q15_sd <- sd(timss$Q15_Index)
Q15_n  <- nrow(timss)
Q15_se <- Q15_sd/sqrt(Q15_n)
Q15_se
```

```
## [1] 0.1105471
```

However, we can also use the `describe()` function from the `psych` package to do this work for us.

Let's pull the `psych` package from our "library" of packages into our current session so we can use its functions and then put it to use:

```
library(psych)
describe(timss$Q15_Index)
```

```
##      vars  n mean   sd median trimmed  mad  min max range skew kurtosis   se
## X1      1 30 1.72 0.61   1.57   1.68 0.64 0.57   3  2.43 0.43   -0.62 0.11
```

We can do this with multiple variables at the same time and create a report:

¹⁸If there were missing data, we would specify `na.rm = TRUE` for the `sd()` function and use `Q15_n <- sum(!is.na(Q15_Index))` for the n -size calculation, which is a function that counts the number of observations that are not missing.

```
report1 <- describe(timss[,c("Q15_Index",
                             "Q16_Index",
                             "Q19_Index",
                             "Q13_NumSs",
                             "Q17_Mins")])

report1
```

```
##          vars  n   mean    sd median trimmed   mad    min    max range  skew
## Q15_Index    1 30   1.72  0.61   1.57    1.68  0.64   0.57   3.00  2.43  0.43
## Q16_Index    2 30   0.72  0.27   0.71    0.71  0.21   0.14   1.43  1.29  0.39
## Q19_Index    3 30   1.56  0.83   1.83    1.57  0.74   0.00   3.00  3.00 -0.22
## Q13_NumSs    4 30  23.13  5.75  24.00   23.46  4.45   8.00  35.00 27.00 -0.51
## Q17_Mins     5 30 193.67 29.30 190.00  192.92 29.65 150.00 240.00 90.00  0.31
##          kurtosis  se
## Q15_Index    -0.62 0.11
## Q16_Index     0.14 0.05
## Q19_Index    -1.27 0.15
## Q13_NumSs     0.14 1.05
## Q17_Mins     -1.25 5.35
```

This is great. We have a lot of statistics here that we can include in our write-up. One thing we need, however, that is not included is the confidence interval of the estimated mean. We can create two new columns for this, the lower bound and upper bound of the interval using the formula $mean \pm z(SE_{mean})$ or $mean \pm t(SE_{mean})$ depending on whether we want to use a z - or a t -distribution.

However, I stumbled into a problem. The psych package seems to assign attributes to objects we create with the `describe()` function, which I found was causing a problem when I wanted to perform operations and create new variables in the data frame. I am adding this code here because my code below (which worked in earlier versions of the psych package) was throwing me an error. In looking at the outputted object's type, which we could see with the `str()` function or more clearly with the `class()` function, we see that this is not just a data frame, which is probably the source of my coding problems when I tried to add a new column:

```
class(report1)
```

```
## [1] "psych"      "describe"   "data.frame"
```

This shows that the object is not just a data frame. Let's coerce to be *just* a data frame by using the `data.frame()` function on it:

```
report1 <- data.frame(report1)
class(report1)
```

```
## [1] "data.frame"
```

That's better. I wish the psych package were friendlier. In this next section, we'll calculate the confidence interval and attach it to this report data frame.

4.2.1 Calculating the confidence interval

We can calculate the estimated 95% confidence interval from the information from the report. Let's use a t -distribution to multiply by the standard error. This requires the degrees of freedom, which is $n - 1$ here. The `qt()` function can be used to get this distribution with the 30 respondents:

```
qt(.975, df = 29)
```

```
## [1] 2.04523
```

This is one tail of the distribution. The other tail would be the negative of this, -2.0452 . If we wanted to use the normal (z) distribution, we can use the `qnorm()` function:

```
qnorm(.975)
```

```
## [1] 1.959964
```

The t distribution is more conservative, which increases the credibility of our inferences about the confidence interval, though it does result in a broader confidence interval than with the z distribution.

With the first variable, the confidence interval is this (with some rounding error):¹⁹

```
1.72 - qt(0.975, df = 30 - 1)*0.11
```

```
## [1] 1.495025
```

```
1.72 + qt(0.975, df = 30 - 1)*0.11
```

```
## [1] 1.944975
```

4.2.2 Generating and exporting a report

We can use the `dplyr` package to create the new variables and reorder the variables. The `%>%` symbol means that we take the object before it, `report1`, and do the following things. Here, we're using the `mutate()` function to create new variables for the upper and lower 95% confidence interval boundaries. We also use `select()` to select which variables we want in our final data set and their order.

```
# install.packages("dplyr") # Run this line if you need to install it
library(dplyr)
MyReport1 <- report1 %>%
  mutate(LB_t = mean - qt(0.975, df = n - 1)*se,
         UB_t = mean + qt(0.975, df = n - 1)*se,
         varb = rownames(.) ) %>%
  select(varb, n, mean, sd, min, max, se, LB_t, UB_t)
MyReport1
```

¹⁹A shorter code for these two lines is `1.72 + c(-1, 1) * qt(0.975, df = 30 - 1)*0.11`.

```
##      varb  n      mean      sd      min      max      se
## 1 Q15_Index 30  1.7190476 0.6054913 0.5714286 3.000000 0.11054709
## 2 Q16_Index 30  0.7190476 0.2717870 0.1428571 1.428571 0.04962129
## 3 Q19_Index 30  1.5555556 0.8319913 0.0000000 3.000000 0.15190013
## 4 Q13_NumSs 30 23.1333333 5.7519612 8.0000000 35.000000 1.05015963
## 5 Q17_Mins 30 193.6666667 29.3002687 150.0000000 240.000000 5.34947271
##      LB_t      UB_t
## 1  1.4929534  1.9451418
## 2  0.6175607  0.8205345
## 3  1.2448849  1.8662262
## 4 20.9855157 25.2811509
## 5 182.7257665 204.6075668
```

We can export this to a CSV file, open it in a spreadsheet program like Excel, format the numbers, and then copy and paste the cells into a table in Word or some other word processor.

```
write.csv(MyReport1,"report1.csv", row.names = FALSE)
```

4.3 But really, it's inferential

This chapter introduced some ways to calculate and report descriptive statistics. We included in our reports the standard error and the confidence intervals around the mean. However, strictly speaking, any time we seek to draw inferences about a larger population, we are actually estimating *inferential* statistics, because—well—we are drawing an inference about a larger group of people (the population) based on our sample.²⁰ Whenever we estimate a statistic, such as the mean (or the variance), we have a point estimate. It is very unlikely that this point estimate is the exact mean that is present in the population. The confidence intervals help us to judge how well we can infer from our actual data what the population mean is.

5 Descriptive Statistics by Group

5.1 Using the describeBy() function

We can get descriptive statistics by group using the psych package's `describeBy()` function. In this case, we are interested in two groups: whether the teachers majored in education or not. In this chapter, we will estimate these, export them to a report, and plot the by-group data for one of the variables.

In the previous chapter, we had created a factor using this code:

```
timss$Q6_Major_in_Edu_F <- factor(timss$Q6_Major_in_Edu,
                                levels = c(0,1),
                                labels = c("Not_education",
                                           "Education") )
```

²⁰The reason for this discrepancy seems to be that in many studies in education and psychology, our research questions are about differences between groups, changes over time, or the relationship among variables, such as in a regression. We perform inferential statistical analyses to answer these questions—things like *t*-tests, repeated-measures ANOVA (including paired *t*-tests) or regression, and estimates of correlation or regression parameters. Traditionally, before we present those inferential-statistics results, we tend to present the descriptive statistics of the groups or time points we are measuring. In those scenarios, they are called descriptive statistics and are not intended to address inferential questions about the population because that part is reserved for our research questions. In contrast, in survey methods, our research questions might simply be about the mean of the population, in which case, the confidence intervals around those means help us to draw conclusions about how precise our estimates of the means are.

We will use this column in the data frame as our by-variable.

We can see that the `describeBy()` function is very similar to the `describe()` function, except in its name and in the addition of the `group =` argument:

```
library(psych)
report2 <- describeBy(timss[,c("Q15_Index",
                                "Q16_Index",
                                "Q19_Index",
                                "Q13_NumSs",
                                "Q17_Mins")],
                      group = timss$Q6_Major_in_Edu_F)
```

Let's take a look at the output:

```
report2
```

```
##
## Descriptive statistics by group
## group: Not_education
##      vars  n   mean    sd median trimmed   mad    min    max range  skew
## Q15_Index   1 13   1.24  0.24   1.29   1.27  0.21   0.57   1.57  1.00 -1.33
## Q16_Index   2 13   0.88  0.27   0.86   0.86  0.21   0.57   1.43  0.86  0.44
## Q19_Index   3 13   1.95  0.73   2.00   2.06  0.49   0.00   2.67  2.67 -1.44
## Q13_NumSs   4 13  25.77  4.71  26.00  25.64  4.45  18.00  35.00 17.00  0.09
## Q17_Mins    5 13 182.31 23.86 170.00 180.91 29.65 150.00 230.00 80.00  0.60
##      kurtosis  se
## Q15_Index     1.79 0.07
## Q16_Index    -1.00 0.07
## Q19_Index     1.27 0.20
## Q13_NumSs    -0.81 1.31
## Q17_Mins    -0.87 6.62
## -----
## group: Education
##      vars  n   mean    sd median trimmed   mad    min max range  skew
## Q15_Index   1 17   2.08  0.54   2.00   2.10  0.42   0.86  3  2.14 -0.15
## Q16_Index   2 17   0.60  0.21   0.57   0.60  0.21   0.14  1  0.86 -0.29
## Q19_Index   3 17   1.25  0.80   1.00   1.20  0.99   0.33  3  2.67  0.55
## Q13_NumSs   4 17  21.12  5.78  21.00  21.47  7.41   8.00 29 21.00 -0.62
## Q17_Mins    5 17 202.35 30.73 200.00 203.33 29.65 150.00 240 90.00 -0.05
##      kurtosis  se
## Q15_Index    -0.39 0.13
## Q16_Index    -0.37 0.05
## Q19_Index    -0.91 0.19
## Q13_NumSs    -0.54 1.40
## Q17_Mins    -1.48 7.45
```

5.2 Confidence intervals for each group's estimated mean

For the by-group report, we can copy and paste the code we used in the previous chapter for the confidence intervals, but we have to do this separately with each group's report. So, we need to separate the report into two data frames, estimate the confidence intervals, and then glue them (bind them) back together.

5.2.1 Separating the report by group

The output from the `describeBy()` function we used has “Not_education” and “Education” for the group. We use that to subset the data. For example, to get the descriptive statistics for those teachers who did not major in education, we use this code:

```
report2$Not_education
```

```
##          vars  n   mean    sd median trimmed   mad    min    max range  skew
## Q15_Index    1 13   1.24  0.24   1.29   1.27  0.21   0.57   1.57  1.00 -1.33
## Q16_Index    2 13   0.88  0.27   0.86   0.86  0.21   0.57   1.43  0.86  0.44
## Q19_Index    3 13   1.95  0.73   2.00   2.06  0.49   0.00   2.67  2.67 -1.44
## Q13_NumSs    4 13  25.77  4.71  26.00  25.64  4.45  18.00  35.00 17.00  0.09
## Q17_Mins     5 13 182.31 23.86 170.00 180.91 29.65 150.00 230.00 80.00  0.60
##          kurtosis  se
## Q15_Index        1.79 0.07
## Q16_Index       -1.00 0.07
## Q19_Index        1.27 0.20
## Q13_NumSs       -0.81 1.31
## Q17_Mins       -0.87 6.62
```

Here are the stats for teachers who did major in education:

```
report2$Education
```

```
##          vars  n   mean    sd median trimmed   mad    min max range  skew
## Q15_Index    1 17   2.08  0.54   2.00   2.10  0.42   0.86   3  2.14 -0.15
## Q16_Index    2 17   0.60  0.21   0.57   0.60  0.21   0.14   1  0.86 -0.29
## Q19_Index    3 17   1.25  0.80   1.00   1.20  0.99   0.33   3  2.67  0.55
## Q13_NumSs    4 17  21.12  5.78  21.00  21.47  7.41   8.00  29 21.00 -0.62
## Q17_Mins     5 17 202.35 30.73 200.00 203.33 29.65 150.00 240 90.00 -0.05
##          kurtosis  se
## Q15_Index       -0.39 0.13
## Q16_Index       -0.37 0.05
## Q19_Index       -0.91 0.19
## Q13_NumSs       -0.54 1.40
## Q17_Mins       -1.48 7.45
```

5.2.2 Calculating the confidence interval

Let’s start with the teachers who did not major in education. Also, notice that there is a new variable that we are creating called `EduMajor`. For this group, we set it to “No”.

```
library(dplyr)
report2_0 <- report2$Not_education %>%
  mutate(EduMajor = "No",
         LB_t = mean - qt(0.975, df = n - 1)*se,
         UB_t = mean + qt(0.975, df = n - 1)*se,
         varb = rownames(.)) %>%
  data.frame() %>% # have to add this because the psych package messed up the format (described in the
  select(varb, EduMajor, n, mean, sd, min, max, se, LB_t, UB_t)
```

Now, for the education majors:

```
report2_1<- report2$Education %>%
  mutate(EduMajor = "Yes",
         LB_t = mean - qt(0.975, df = n -1)*se,
         UB_t = mean + qt(0.975, df = n -1)*se,
         varb = rownames(.)) %>%
  data.frame() %>%
  select(varb, EduMajor, n, mean, sd, min, max, se, LB_t, UB_t)
```

We can stack these two data frames on top of each other with the `rbind()` function, which binds the rows together. We can do this because the two different data frames have precisely the same column names and types of data.

```
MyReport2 <- rbind(report2_0, report2_1)
MyReport2
```

##	varb	EduMajor	n	mean	sd	min	max
## 1	Q15_Index	No	13	1.2417582	0.2431694	0.5714286	1.571429
## 2	Q16_Index	No	13	0.8791209	0.2662805	0.5714286	1.428571
## 3	Q19_Index	No	13	1.9487179	0.7308817	0.0000000	2.666667
## 4	Q13_NumSs	No	13	25.7692308	4.7108712	18.0000000	35.000000
## 5	Q17_Mins	No	13	182.3076923	23.8585576	150.0000000	230.000000
## 6	Q15_Index	Yes	17	2.0840336	0.5417725	0.8571429	3.000000
## 7	Q16_Index	Yes	17	0.5966387	0.2096845	0.1428571	1.000000
## 8	Q19_Index	Yes	17	1.2549020	0.7952062	0.3333333	3.000000
## 9	Q13_NumSs	Yes	17	21.1176471	5.7758371	8.0000000	29.000000
## 10	Q17_Mins	Yes	17	202.3529412	30.7264975	150.0000000	240.000000
##	se	LB_t	UB_t				
## 1	0.06744305	1.0948125	1.3887040				
## 2	0.07385291	0.7182092	1.0400326				
## 3	0.20271011	1.5070506	2.3903853				
## 4	1.30656060	22.9224798	28.6159818				
## 5	6.61717328	167.8901103	196.7252744				
## 6	0.13139912	1.8054799	2.3625873				
## 7	0.05085596	0.4888288	0.7044485				
## 8	0.19286584	0.8460446	1.6637593				
## 9	1.40084626	18.1479857	24.0873085				
## 10	7.45227027	186.5548339	218.1510484				

We can use the `arrange()` function from the `dplyr` package to order the data by their variables. This way we can see the estimates of each group on a particular variable in adjacent rows.

```
library(dplyr)
MyReport2 <- arrange(MyReport2, varb)
MyReport2
```

##	varb	EduMajor	n	mean	sd	min	max
## 1	Q13_NumSs	No	13	25.7692308	4.7108712	18.0000000	35.000000
## 2	Q13_NumSs	Yes	17	21.1176471	5.7758371	8.0000000	29.000000
## 3	Q15_Index	No	13	1.2417582	0.2431694	0.5714286	1.571429
## 4	Q15_Index	Yes	17	2.0840336	0.5417725	0.8571429	3.000000

```
## 5 Q16_Index      No 13  0.8791209  0.2662805  0.5714286  1.428571
## 6 Q16_Index      Yes 17  0.5966387  0.2096845  0.1428571  1.000000
## 7 Q17_Mins       No 13 182.3076923 23.8585576 150.0000000 230.000000
## 8 Q17_Mins       Yes 17 202.3529412 30.7264975 150.0000000 240.000000
## 9 Q19_Index      No 13  1.9487179  0.7308817  0.0000000  2.666667
## 10 Q19_Index     Yes 17  1.2549020  0.7952062  0.3333333  3.000000
##          se          LB_t          UB_t
## 1  1.30656060 22.9224798 28.6159818
## 2  1.40084626 18.1479857 24.0873085
## 3  0.06744305  1.0948125  1.3887040
## 4  0.13139912  1.8054799  2.3625873
## 5  0.07385291  0.7182092  1.0400326
## 6  0.05085596  0.4888288  0.7044485
## 7  6.61717328 167.8901103 196.7252744
## 8  7.45227027 186.5548339 218.1510484
## 9  0.20271011  1.5070506  2.3903853
## 10 0.19286584  0.8460446  1.6637593
```

Now we have a report with a column that identifies the group. We can export this to a CSV file and then open it with our spreadsheet program to round the numbers to our preferred decimal places before we export it into our word processor's table.

```
write.csv(MyReport2, "report2.csv", row.names = FALSE)
```

5.3 Plotting the by-group means and confidence intervals

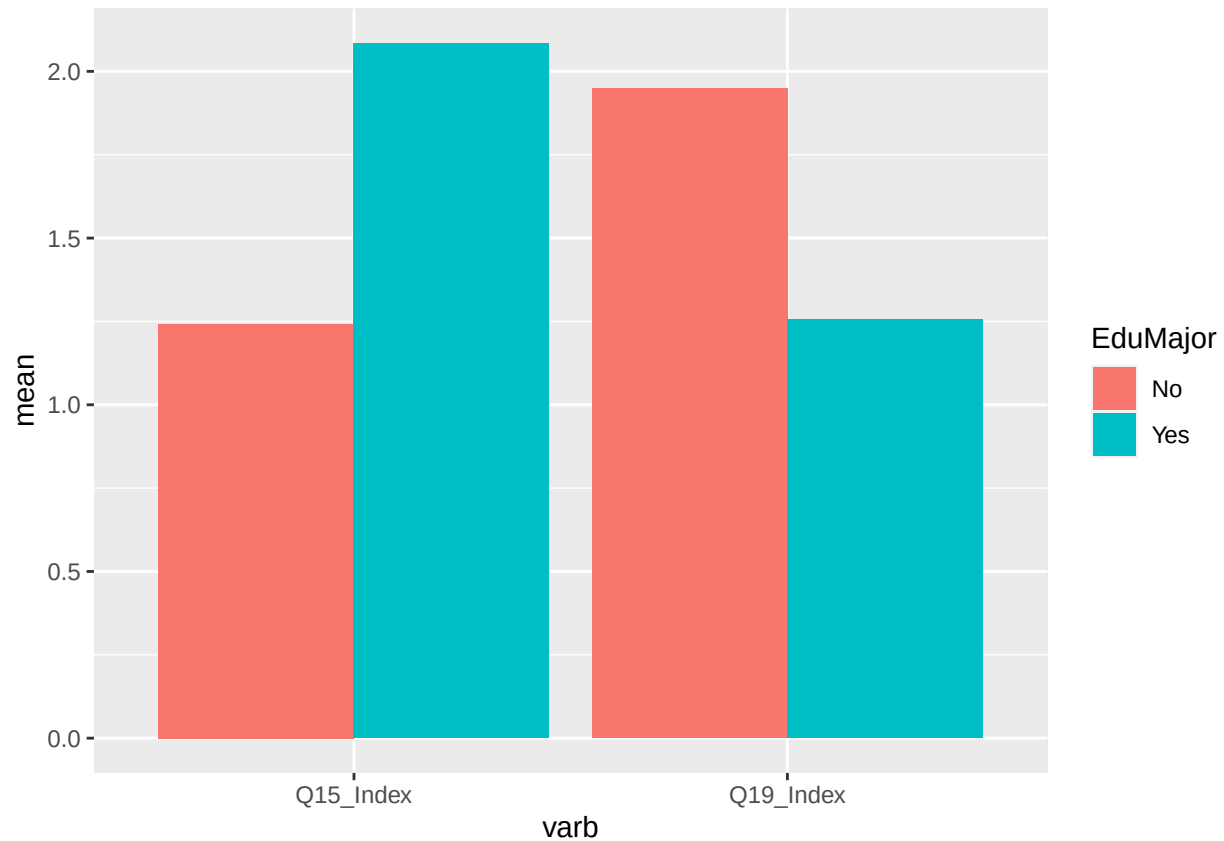
Let's create a plot of two variables side by side: Q15 and Q19, by whether they got their degree in education or not. First, let's subset our data. We can use the `filter()` function from `dplyr`:

```
library(dplyr)
ForPlot <- MyReport2 %>%
  filter(varb == "Q15_Index" | varb == "Q19_Index")
ForPlot
```

```
##          varb EduMajor  n    mean      sd      min      max      se
## 1 Q15_Index      No 13 1.241758 0.2431694 0.5714286 1.571429 0.06744305
## 2 Q15_Index      Yes 17 2.084034 0.5417725 0.8571429 3.000000 0.13139912
## 3 Q19_Index      No 13 1.948718 0.7308817 0.0000000 2.666667 0.20271011
## 4 Q19_Index      Yes 17 1.254902 0.7952062 0.3333333 3.000000 0.19286584
##          LB_t      UB_t
## 1 1.0948125 1.388704
## 2 1.8054799 2.362587
## 3 1.5070506 2.390385
## 4 0.8460446 1.663759
```

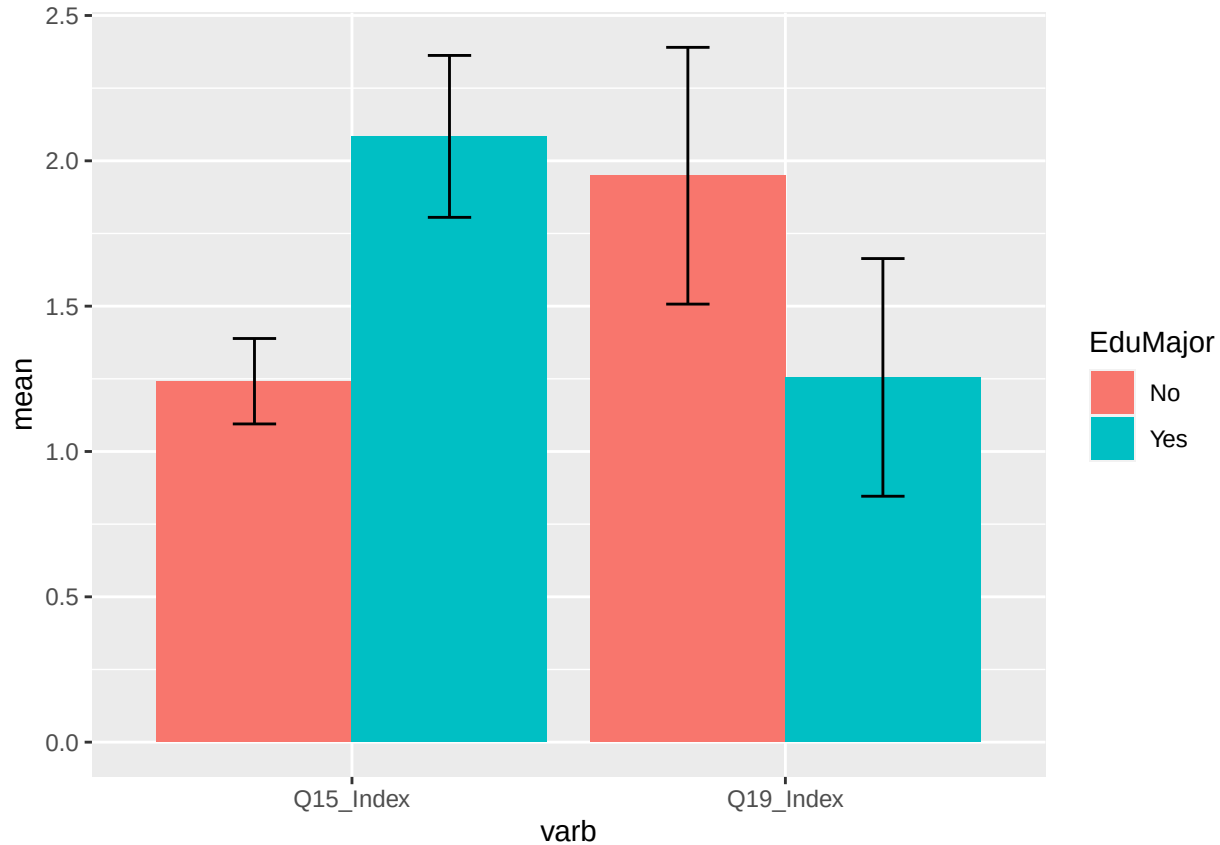
Now, we have a data frame of our report that contains only these two variables. Let's use the `ggplot2` package and a geom called `geom_col`, which makes columns and allows us to position groups' scores adjacent to each other (with the `position = "dodge"` argument):

```
library(ggplot2)
ggplot(data = ForPlot, aes(x = varb, y = mean, fill=EduMajor)) +
  geom_col(position = "dodge")
```



Let's add confidence intervals to the plot. The `geom_errorbar()` part of the code displays a band around the mean, which is the top of our bars. This allows us to eyeball whether the two groups are likely to be statistically different or not. Notice that we are using the lower bound and upper bound estimates that we had calculated earlier.

```
Myplot <- ggplot(ForPlot, aes(x = varb, y = mean, fill=EduMajor)) +
  geom_col(width = 0.9, position = "dodge") +
  geom_errorbar(aes(ymin = LB_t, ymax = UB_t),
    position = position_dodge(0.9), width = 0.2)
Myplot
```



This plot says a lot. The two groups clearly differ on the Q15_Index. Their confidence intervals don't overlap at all. On the Q19_Index, they might still statistically significantly differ. With 95% confidence intervals that overlap less than about 25%, we can still observe statistically significant difference.²¹

5.3.1 Making a pretty plot, maybe

The following code generates a plot that is more informative and perhaps more pleasing in format.²² Notice that the scale of the composite scores is the same as that used in the items. This helps us interpret the meaning of these composite scores. For instance, with the teachers who majored in education when they were in college, the estimated mean on the Q15 index is 2.08, which slightly more frequent than *About half the lessons* on the original Likert-type scale.

```
MyPretty <- Myplot +
  theme_grey(base_size = 16) +
  theme(axis.title.x.bottom=element_blank() ) +
  scale_fill_viridis_d(option = "cividis", begin=0.2,end=0.6) +
  ylab("Mean score") +
  scale_x_discrete(labels=c("Student centered\nteaching approach",
                           "Didactic\nteaching approach")) +
  guides(fill = FALSE) +
  annotate("text", x = .65, y = 0.10, size = 5, color="white",
         label = "Did not major\nin education",
```

²¹Note that it is a misconception that statistically significant different results only occur when confidence intervals do not overlap. Nope. Not true.

²²The viridis color palette is included in this plot. Viridis color palettes are designed to be a little more accessible to people with colorblindness.

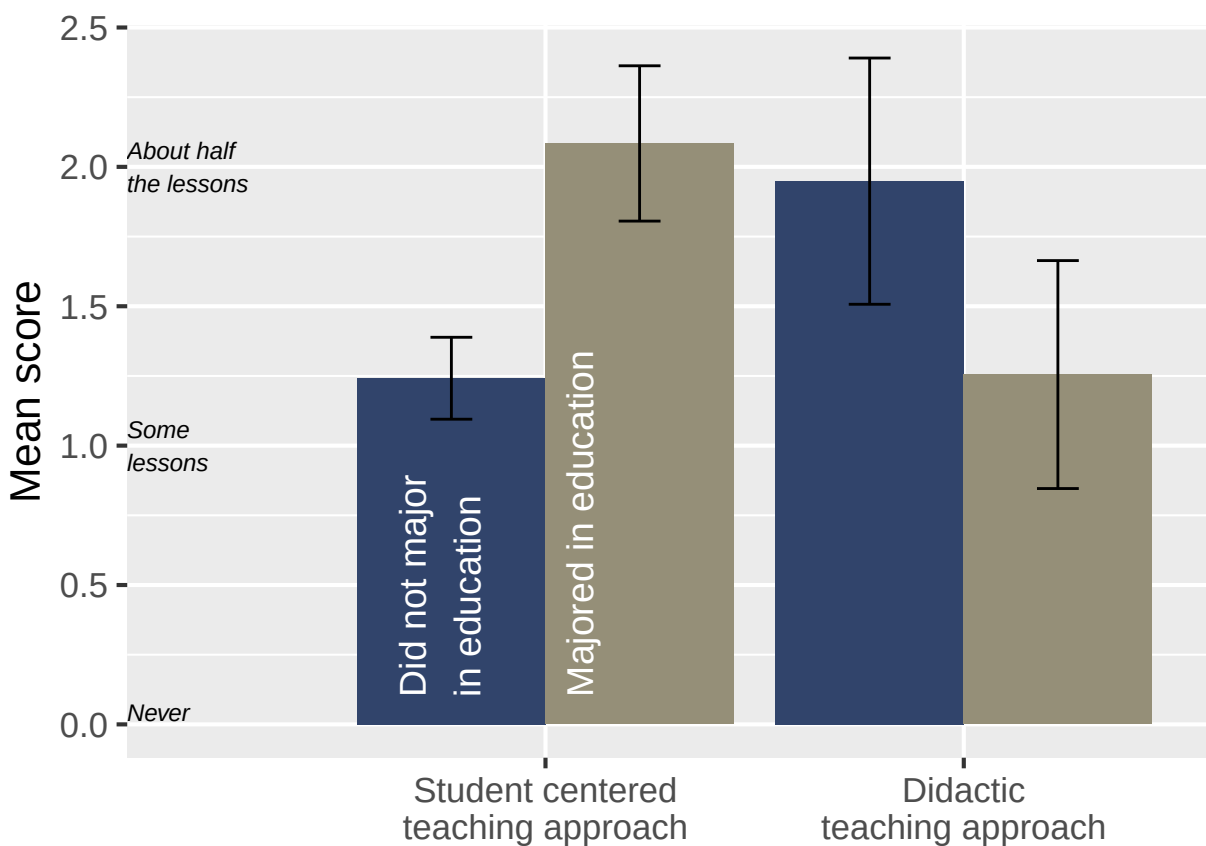
```

        hjust = 0, vjust=1, angle = 90) +
annotate("text", x = 1.05, y = 0.10, size = 5, color="white",
        label = "Majored in education",
        hjust = 0, vjust=1, angle = 90) +

annotate("text", x = 0, y = 0, fontface="italic", size = 3,
        label = "Never", hjust=0, vjust=-.2) +
annotate("text", x = 0, y = 1, fontface="italic", size = 3,
        label = "Some\nlessons", hjust=0) +
annotate("text", x = 0, y = 2, fontface="italic", size = 3,
        label = "About half\nthe lessons", hjust=0)

```

MyPretty



5.3.2 Saving plots

We can save plots to objects and then export them to .png or .pdf formats. The image, in .png format, is easy to paste into a word processor slide show for your report:

```

ggsave("MyPrettyPlot.png", MyPretty, width = 6, height = 5)
ggsave("MyPrettyPlot.pdf", MyPretty, width = 6, height = 5)

```

6 Internal Consistency Reliability

6.1 Doing it manually

Let's hand calculate it for one of our scales, Q15, using the formula for coefficient alpha in the slides. Here's the covariance matrix:²³

```
round(cov(timss[,Q15]),2)
```

```
##           Q15_StCentrd_a Q15_StCentrd_b Q15_StCentrd_c Q15_StCentrd_d
## Q15_StCentrd_a           0.74           0.32           0.04           0.51
## Q15_StCentrd_b           0.32           0.56          -0.02           0.26
## Q15_StCentrd_c           0.04          -0.02           0.84           0.21
## Q15_StCentrd_d           0.51           0.26           0.21           0.85
## Q15_StCentrd_e           0.50           0.36          -0.05           0.36
## Q15_StCentrd_f           0.30           0.28           0.22           0.20
## Q15_StCentrd_g           0.55           0.41           0.16           0.60
##           Q15_StCentrd_e Q15_StCentrd_f Q15_StCentrd_g
## Q15_StCentrd_a           0.50           0.30           0.55
## Q15_StCentrd_b           0.36           0.28           0.41
## Q15_StCentrd_c          -0.05           0.22           0.16
## Q15_StCentrd_d           0.36           0.20           0.60
## Q15_StCentrd_e           0.62           0.31           0.47
## Q15_StCentrd_f           0.31           0.79           0.41
## Q15_StCentrd_g           0.47           0.41           0.83
```

Here's the total variance. Notice that this is the sum of all of the cells in the variance-covariance matrix:²⁴

```
(totvar <- sum(cov(timss[,Q15])))
```

```
## [1] 17.96437
```

Here's the unique variance of the items:²⁵

```
(itemvarsum <- sum(apply(timss[,Q15],2,var)))
```

```
## [1] 5.224138
```

²³The term *covariance matrix* is an abbreviation of *variance-covariance matrix*, which is more accurate but more cumbersome to say and write. The matrix includes the item variances on the diagonal and the covariances on the off diagonal.

²⁴Note that this total variance calculated as the sum of the matrix's cells is the same as the variance of the composite score when that score is calculated as the sum of the item scores, which is often what we do in tests—we add up each person's points across the items to get their total score. However, to make our composite score easier to interpret on our survey scale, we have calculated each person's score as their mean across the k items making up that scale, as this allows us to see each person's composite score on the same scale as the meaningful points in the Likert-type scale that is used with all k of the items. For example, the first teacher's score on the Q15 composite scale was 2.29, which somewhat more frequent than *About half the lessons* on the original Likert-type scale. That is easier to interpret than a sum score would be—it is harder to relate the person's sum score of 16.03 back to a meaningful scale. Because our scale score is now divided by the number of items ($k = 7$ in this case), this total variance differs from that of the sum of the variance-covariance cells—it is smaller because we have divided by the constant 7. By the rules of algebra of expectation, we can multiply the variance of this, which is `var(timss$Q15_Index) = 0.37`, by this constant squared to retrieve the same variance as if the composite score were calculated as the sum across the items: `var(timss$Q15_Index)*ncol(timss[,Q15])^2 = 17.96`. This value is the same as the sum of the cells in the covariance matrix.

²⁵Placing parentheses around R code forces it to be printed to the console. It's simply a shortcut so we don't have to type the object (`totvar` in this example) to view the result.

Here's k , which is the number of items in the scale:

```
(k <- ncol(timss[,Q15]))
```

```
## [1] 7
```

And, here's the alpha coefficient:

```
MyAlpha <- k/(k-1)*(totvar - itemvarsum)/totvar
MyAlpha
```

```
## [1] 0.8273935
```

6.2 Doing it with the psych package

```
library(psych)
# help(package = "psych")
```

To get coefficient alpha, we use the `alpha()` function.²⁶ Take a look at the `raw_alpha` column under the `Reliability if an item is dropped` section. What is this? Note where it is reported in our Word document that was shared on Laulima. In looking at this particular part of the output, what can we conclude about Item Q15_StCentrd_c?²⁷

```
psych::alpha(timss[,Q15])
```

```
##
## Reliability analysis
## Call: psych::alpha(x = timss[, Q15])
##
##   raw_alpha std.alpha G6(smc) average_r S/N   ase mean   sd median_r
##      0.83      0.83    0.86      0.41 4.9 0.048   2.3 0.61      0.44
##
##   lower alpha upper      95% confidence boundaries
## 0.73 0.83 0.92
##
## Reliability if an item is dropped:
##      raw_alpha std.alpha G6(smc) average_r S/N alpha se var.r med.r
## Q15_StCentrd_a    0.78    0.78    0.82    0.38 3.6   0.062 0.055 0.42
## Q15_StCentrd_b    0.81    0.81    0.85    0.41 4.2   0.055 0.064 0.44
## Q15_StCentrd_c    0.87    0.87    0.88    0.53 6.9   0.036 0.021 0.50
## Q15_StCentrd_d    0.79    0.80    0.83    0.40 4.0   0.059 0.069 0.44
## Q15_StCentrd_e    0.79    0.79    0.82    0.39 3.8   0.059 0.051 0.40
## Q15_StCentrd_f    0.81    0.82    0.85    0.43 4.5   0.052 0.079 0.49
## Q15_StCentrd_g    0.76    0.77    0.81    0.36 3.3   0.069 0.057 0.40
##
## Item statistics
```

²⁶Here, we're using the double colon before the function because the function `alpha()` shares the same name with an argument in the `ggplot` package, so this `psych::alpha()` ensures that the function comes from the `psych` package.

²⁷The answer is that the item is functioning inconsistently with the other items in the scale. We can see the reliability of the scale would be *higher* if we had eliminated the item from our calculation of the composite score.

```
##               n raw.r std.r r.cor r.drop mean  sd
## Q15_StCentrd_a 30  0.81  0.81  0.80  0.72  2.4 0.86
## Q15_StCentrd_b 30  0.68  0.71  0.64  0.57  1.8 0.75
## Q15_StCentrd_c 30  0.36  0.34  0.19  0.15  2.3 0.92
## Q15_StCentrd_d 30  0.76  0.75  0.72  0.64  2.3 0.92
## Q15_StCentrd_e 30  0.77  0.79  0.77  0.68  2.3 0.78
## Q15_StCentrd_f 30  0.66  0.66  0.58  0.52  2.6 0.89
## Q15_StCentrd_g 30  0.88  0.88  0.89  0.82  2.2 0.91
##
## Non missing response frequency for each item
##           1      2      3      4 miss
## Q15_StCentrd_a 0.13 0.40 0.37 0.10   0
## Q15_StCentrd_b 0.37 0.43 0.20 0.00   0
## Q15_StCentrd_c 0.23 0.30 0.40 0.07   0
## Q15_StCentrd_d 0.23 0.27 0.43 0.07   0
## Q15_StCentrd_e 0.13 0.53 0.27 0.07   0
## Q15_StCentrd_f 0.13 0.23 0.50 0.13   0
## Q15_StCentrd_g 0.27 0.37 0.30 0.07   0
```

Does the estimate for alpha match our hand calculation? It should (though there the output here is rounded to two decimal places).

What's the internal consistency reliability of the other two composite scales (AKA indexes)?

6.2.1 Omega reliability using the psych package

For omega reliability use the omega function. Because we have only one score, the number of factors is 1 and we report the result in Omega Total. Omega is appropriate for reflective scoring models, which was described in the validity PDF.

```
psych::omega(timss[,Q15], nfactors=1)
```

```
## Omega
## Call: omegah(m = m, nfactors = nfactors, fm = fm, key = key, flip = flip,
##   digits = digits, title = title, sl = sl, labels = labels,
##   plot = plot, n.obs = n.obs, rotate = rotate, Phi = Phi, option = option,
##   covar = covar)
## Alpha:                0.83
## G.6:                  0.86
## Omega Hierarchical:    0.85
## Omega H asymptotic:    1
## Omega Total            0.85
##
## Schmid Leiman Factor loadings greater than 0.2
##           g  F1*   h2   u2 p2
## Q15_StCentrd_a 0.83    0.69 0.31 1
## Q15_StCentrd_b 0.66    0.44 0.56 1
## Q15_StCentrd_c    0.02 0.98 1
## Q15_StCentrd_d 0.69    0.47 0.53 1
## Q15_StCentrd_e 0.80    0.65 0.35 1
## Q15_StCentrd_f 0.53    0.28 0.72 1
## Q15_StCentrd_g 0.90    0.81 0.19 1
##
```

```

## With eigenvalues of:
##   g F1*
## 3.3 0.0
##
## general/max 6.025027e+16   max/min =    1
## mean percent general =    1   with sd =    0 and cv of    0
## Explained Common Variance of the general factor =    1
##
## The degrees of freedom are 14   and the fit is    0.65
## The number of observations was    30   with Chi Square =    16.25   with prob <    0.3
## The root mean square of the residuals is    0.09
## The df corrected root mean square of the residuals is    0.11
## RMSEA index =    0.065   and the 10 % confidence intervals are    0 0.202
## BIC =   -31.37
##
## Compare this with the adequacy of just a general factor and no group factors
## The degrees of freedom for just the general factor are 14   and the fit is    0.65
## The number of observations was    30   with Chi Square =    16.25   with prob <    0.3
## The root mean square of the residuals is    0.09
## The df corrected root mean square of the residuals is    0.11
##
## RMSEA index =    0.065   and the 10 % confidence intervals are    0 0.202
## BIC =   -31.37
##
## Measures of factor score adequacy
##
##                                     g F1*
## Correlation of scores with factors      0.96    0
## Multiple R square of scores with factors      0.92    0
## Minimum correlation of factor score estimates 0.84   -1
##
## Total, General and Subset omega for each subset
##
##                                     g F1*
## Omega total for total scores and subscales      0.85 0.85
## Omega general for total scores and subscales      0.85 0.85
## Omega group for total scores and subscales      0.00 0.00

```

If you came across an error that said "Failed with error: 'there is no package called 'GPArotation''", install that package and run the previous line again.

```
install.packages("GPArotation")
```

Look at the bottom part of the output. We are not working with more than one factor, or subscale, so we can report the estimate under the *g* column (“g” stands for general). The first or second line, “Omega general for total scores and subscales” under *g* is **.85**. That’s what we can report for this scale.

6.3 Summary

There are several types of reliability coefficients. Coefficients alpha and omega are for internal consistency. Coefficient alpha is the most popular. Coefficient omega is becoming more popular because it handles some of the problems that alpha ignores.

Other coefficients, such as for estimates of stability over time, with the intra-class correlation and generalizability coefficients are addressed in measurement primers and in EDEP 616.

7 Introduction to the Analysis of Complex Survey Data

In a garden variety simple-random-sample design, we randomly sample units from a larger population and then estimate the mean, standard deviation, and confidence intervals to subsequently draw an inference about the larger population. This is the assumption underlying the example and explanations in the previous chapters. In this chapter we examine complex survey data. In survey designs with complex data, we need to make corrections to the standard error, which in turn affects the confidence intervals, which subsequently affects our inference about the statistic—such as the mean—in the population we seek to make claims about.

7.1 Packages

There are two packages that are handy for analyzing complex survey data: The survey package [Lumley, 2020] and the similarly named srvyr package [Freedman Ellis and Schneider, 2021], which works with the survey package to make the output easier to work with (because why would the survey package on its own make life easy?). Harkening back to a previous chapter, we need to first be sure we have these packages installed. We will also use the dplyr package [Wickham et al., 2020b] again, as this uses the same data-wrangling language that the srvyr package does.²⁸

```
library(survey)
library(srvyr)
library(dplyr)
```

7.2 Example data

To go through this example, let's use the data set that is presented in the chapter by Heeringa, West, and Berglund [2010, Chapter 2], who provide an explanation that we can follow along with as we actually perform these calculations.²⁹

```
dat <- read.csv("Heeringa.Complex.SampleData.csv")
str(dat)
```

```
## 'data.frame':    32 obs. of  5 variables:
## $ sid      : int  1 2 3 4 5 6 7 8 9 10 ...
## $ school   : int  1 1 1 1 2 2 2 2 3 3 ...
## $ district: int  1 1 1 1 1 1 1 1 2 2 ...
## $ score    : num  52.8 32.5 56.6 47 37.3 57 54.2 71.5 27.7 42.3 ...
## $ weight   : int  1 2 2 1 1 1 2 2 1 2 ...
```

```
head(dat)
```

```
##   sid school district score weight
## 1    1     1         1  52.8      1
## 2    2     1         1  32.5      2
## 3    3     1         1  56.6      2
## 4    4     1         1  47.0      1
## 5    5     2         1  37.3      1
## 6    6     2         1  57.0      1
```

²⁸Whenever we use a package, it is a good idea to cite it. People work hard to create these (well usually—for some, it is disgustingly easy). To get the citation, we can use the `citation()` function, usually, as in `citation("srvyr")`.

²⁹In their chapter, their variable of interest is persons' ratings of the economy. We use the same data but for demonstration call the outcome variable *scores* and label the cases *students*, who are nested within the school clusters, which are in districts, serving as strata. For this demo, we will ignore weighting.

7.3 The three hypothetical complex designs

7.3.1 Pretending we have a cluster-sampled design

We will use this data set to practice analyzing the data in a cluster-sampled design. This would be the kind of analysis we would conduct if we had randomly sampled schools—that is, our clusters—from a broader population of schools (such as from a sampling frame that listed all of the schools in the country). In this design, our *primary sampling unit* is schools. Within each school, we have students, whom we had administered the survey to.³⁰ This design is very convenient because it is easier to randomly sample schools from a list of schools, whereas it is virtually impossible to have a list of all of the individual students in the country. We pay a price for this convenience, though, because students who are nested within schools tend to live in similar neighborhoods, share the same values, have the same learning experiences, and so forth, so if we used a run-of-the-mill standard error (and confidence interval), we will be overly optimistic in how precise we are in estimating the population. When we correctly analyze these data, we pay a penalty because our standard error is corrected to be larger than it would be with a simple random sample.

7.3.2 Pretending we have a stratified design

After analyzing the cluster-sampling design data, we'll erase our memories of that design and pretend we're analyzing a stratified design. In this type of design, we have conducted a simple random sample of students within strata. In this case, the students are our *primary sampling unit*. Let's say we have a list of school districts—these are our strata. We then have a list of all the students in each district, which is our sampling frame. We randomly selected students from each district, then we estimate the mean of the population. This is a very powerful design if we believe that the strata explain a lot of variation in the variable (in the score, in this case). It helps us to have greater precision in our estimate of the population mean.³¹

7.3.3 Pretending we have a stratified cluster-sampled design

After monkeying around with a stratified design, we wipe our memories of how we had designed that study and pretend we had collected the data using schools as our primary sampling unit, similar to how we did with the cluster-sampled design, but this time we sample schools from within their districts. The districts are strata. So, we have students nested within schools, which were randomly selected, which in turn are nested within strata. This is a very practical and smart design to use. First, we have the convenience of sampling schools, which is far easier than randomly sampling students from lists. Second, we can reduce that penalty to the standard error of this clustering by also using a stratification variable—the districts. This allows us to make claims about the population of schools within these particular districts.

We will also pretend we have a simple random sample, as well, to compare these scenarios with each other in terms of the standard error and confidence intervals.

7.4 Cluster-sampled design

7.4.1 Preparing the data

In the example that Heeringa and colleagues provide, they did not use a finite population correction. Another way to think about this is that the population of our primary sampling unit is very large. Let's say, therefore, that there is a very large number of schools (such as one million) in the population.

³⁰Before we got to this step, we had previously calculated each student's score, which is the `score` variable in our data set. This score might be a composite score calculated as the mean (or sum) across several items, as was demonstrated in the previous chapter.

³¹Strata serve as a kind of fixed effect or a blocking variable in ANOVA.

For many researchers, they treat the population as an infinitely large number, which is a conservative approach because it does not adjust the standard error to be smaller to account for the proportion of the population being sampled. For other researchers, such as for evaluations in a specific population—such as the schools in a particular state—the population size is finite. For instance, if there are 160 schools in a state and we’re only drawing inferences about schools in that state, we can use this number as our population size, which will yield a lower standard error (which will narrow our confidence intervals compared to an infinite population size).

The following example treats the population as infinite (to follow Heeringa’s example), but in your own study, you can specify these sizes to match your needs in your research or evaluation.

To get this to work with the survey package, we create a column in the data that is this cluster N -size. Here, we can use `N_cl` as this column name to remind us that this is the population size of clusters.

```
dat$N_cl <- 1000000
head(dat)
```

```
##   sid school district score weight  N_cl
## 1    1      1         1  52.8      1 1e+06
## 2    2      1         1  32.5      2 1e+06
## 3    3      1         1  56.6      2 1e+06
## 4    4      1         1  47.0      1 1e+06
## 5    5      2         1  37.3      1 1e+06
## 6    6      2         1  57.0      1 1e+06
```

7.4.2 Step 1: Specifying the design

To use the `srvyr` package (and the `survey` package it wraps around), we need to create a survey-design object.³² This specifies what the survey design was when we collected the data, which in turn determines what kinds of adjustments to the standard error we should make.

Let’s create a survey-design object, with which we can perform subsequent calculations. We do this with the `srvyr` package’s `as_survey_design()` function. To remind us that this is a design and that it uses clusters, we can name our object `des_cl`, but you could call it `Bill` or `mydesign`.

```
library(survey)
library(srvyr)
des_cl <- dat %>%
  as_survey_design(id = school,
                  fpc = N_cl)
```

There are two arguments in that design specification. The `id =` specifies what the primary sampling unit is. In our design, the primary sampling unit (PSU) is the `school` variable in our data set. The second argument is for the finite population correction. We would think that the `fpc =` argument specifies the actual finite population correction, but it does NOT! Unintuitively, the default in the `survey` package is for `fpc =` to actually refer to the *size* of the population of primary sampling units. So, this should be called something like `pop` instead of `fpc` in the program—that would make it too easy, though, wouldn’t it.³³ If the number of clusters in our population were finite, like 168 schools, we can use the `fpc = 168` as a value for each case in the data set, which would make our standard error smaller. However, for this hypothetical example, we are saying there are 1 million schools.

We can look at the survey design we created using the `summary()` function on that object:

³²For more information on this package, we can use `help(package = "srvyr")` or <https://cran.r-project.org/web/packages/srvyr/vignettes/srvyr-vs-survey.html>.

³³But we can also use the `fptype = "correction"` argument to specify the type of `fpc` discussed in the Heeringa chapter (which was $1 - n/N$). Unfortunately, although we get the correct design effect, the estimated standard error is wrong.

```
summary(des_cl)
```

```
## 1 - level Cluster Sampling design
## With (8) clusters.
## Called via srvyr
## Probabilities:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 8e-06  8e-06   8e-06   8e-06  8e-06   8e-06
## Population size (PSUs): 1e+06
## Data variables:
## [1] "sid"      "school"    "district"  "score"     "weight"    "N_cl"
```

Notice that the probability of being sampled is equal across the quantiles. This is because everyone has the same fpc.

7.4.3 Step 2: Calculating the estimates

We want to know the mean and variance of our sample, as well as the confidence interval of the mean. We cannot use the regular Base R functions like `mean()` and `variance()` because they do not correct for the design. Instead, we use the functions from the `svyr` package. For example, instead of `mean()`, it uses `survey_mean()`.³⁴

```
des_cl %>%
  summarize(survey_mean(score,
                        vartype = c("se", "ci"), df = degf(.),
                        deff = TRUE ))
```

```
## # A tibble: 1 x 5
##   coef `se` `low` `upp` `deff`
##   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  40.8  3.66  32.1  49.4  2.31
```

We have the `summarize()` function around the `survey_mean()` function. Just as we do in mathematics, we work from inside the parentheses to the outside. The first argument in the `survey_mean()` function is the variable that we are estimating, `score` in this case. The second argument specifies the type of sampling-distribution variance we would like to report (not to be confused with the sampling variance). Here, we seek the standard error ("`se`") and the upper and lower 95% confidence interval ("`ci`") based on the t distribution (based on the `df = degf(.)` argument). If we wanted the z distribution, we would remove the `df =` argument. Because t distribution values at the lower and upper 2.5% quantiles are larger in magnitude (depending on the degrees of freedom) than z distribution values (1.96), they are more conservative. We also request the design effect to be reported, `deff = TRUE`.

The coefficient of interest is the mean. The standard error and the lower- and upper-boundaries of the confidence intervals are reported, as well as the design effect.

We can actually save this output to an object and then use it later on. If you are like me, you think the outputs printed from the `srvyr` package are ugly. I want to see two decimal places, so I'll round it to two decimal places, then print it with the `print.data.frame()` function.³⁵

³⁴We're using piping, `%>%` with this particular package just like how we did with the `dplyr` package. These are part of the tidyverse community of R packages.

³⁵We can also use RStudio's `View()` function to see it in an extra tab, with `View(results_M_cl)`.

```

rslts_M_cl <- des_cl %>%
  summarize(Mn = survey_mean(score,
    vartype = c("se","ci"), df = degf(.),
    deff = TRUE ) )
print.data.frame( round(rslts_M_cl, 2) )

```

```

##      Mn Mn_se Mn_low Mn_upp Mn_deff
## 1 40.77  3.66  32.12  49.42    2.31

```

Now let's examine the variance and the sampling distribution of the variance. The function is `survey_var()`:

```

rslts_V_cl <- des_cl %>% summarize(survey_var(score) )
print.data.frame( round(rslts_V_cl, 2) )

```

```

##      coef      _se
## 1 185.28 40.72

```

The coefficient is our estimated variance and the standard error is the standard error of the estimated variance.³⁶

Conveniently, we can perform both the `survey_mean()` and `survey_var()` and save these to a report:

```

rslts_cl <- des_cl %>%
  summarize(Mn = survey_mean(score,
    vartype = c("se","ci"), df = degf(.),
    deff = TRUE ),
    Var= survey_var(score) )
print.data.frame(round(rslts_cl, 2))

```

```

##      Mn Mn_se Mn_low Mn_upp Mn_deff      Var Var_se
## 1 40.77  3.66  32.12  49.42    2.31 185.28 40.72

```

So, we have an estimated mean of the score (whatever construct this happens to be, such as students' judgments of school climate) of 40.77 and a 95% confidence interval of (32.12, 49.42).

7.4.4 Effective sample size

We can calculate the effective sample size from the number of students within the clusters, n , and the design effect. This is helpful for explaining the design effect to folks such as school personnel. First, let's get the n -size of students.

To specify what the *sample* size of our smallest units (students) is, we can count the number of rows in the data using `nrow(dat)`. This is useful if each row is an individual person. We can also use `length(unique(dat$sid))` to count the number of observations that are unique in the student ID column.³⁷ Either way, we get the same number.

³⁶Clinical researchers tend not to report things like standard error of the variance. It is important to realize, however, that any point estimates from a sample, such as the means and variances, are estimates of the population and therefore have some degree of uncertainty, which is what the standard error reveals.

³⁷Sometimes data structures, such as those used in longitudinal analyses, have more than one row for each person. The `unique()` function makes sure we don't count somebody with the same ID twice.

```
n <- length(unique(dat$sid))
n
```

```
## [1] 32
```

As expected, there are 32 units (students) in our sample. Here's the effective sample size, which is calculable as $\frac{n}{\text{design effect}}$.³⁸

```
rslts_cl <- rslts_cl %>% mutate(EffSS = n/Mn_deff)
print.data.frame(round(rslts_cl, 2))
```

```
##      Mn Mn_se Mn_low Mn_upp Mn_deff      Var Var_se EffSS
## 1 40.77  3.66  32.12  49.42    2.31 185.28  40.72 13.84
```

The standard error with this design is about what it would be with a simple random sample of 13.84 (or rounding up, 14) students from the population. This shows the penalizing effect that we have correctly applied to the data.

7.4.5 Reporting the mean per cluster

Sometimes we want to also report the mean per school. We can use the `group_by()` function to specify what the grouping is before we apply the functions. Here, we're grouping by school, then estimating the mean and variance³⁹:

```
M_by_schl <- des_cl %>%
  group_by(school) %>%
  summarize(Mn = survey_mean(score),
            Var= survey_var(score))
print.data.frame(round(M_by_schl,2))
```

```
##   school    Mn Mn_se    Var Var_se
## 1      1 47.23    0 111.95    0
## 2      2 55.00    0 196.73    0
## 3      3 34.40    0  37.71    0
## 4      4 52.22    0 151.26    0
## 5      5 35.65    0 216.99    0
## 6      6 43.55    0  82.89    0
## 7      7 25.50    0 100.05    0
## 8      8 32.60    0  17.19    0
```

7.4.6 Two-stage cluster-sampled design

The method so far has assumed that the units within the clusters were the population within those clusters. For example, if we surveyed all of the students in our selected schools. In a two-stage design, we have the extra step of randomly selecting units within the clusters.

³⁸The code below uses the `mutate()` function. This is from the `dplyr` package (for data wrangling) and is the same as this code: `rslts_cl$EffSS = n/rslts_cl$Mn_deff`. The benefit of `mutate()` is we can create multiple variables in the mutate function and we don't have to specify the data frame, such as with `rslts_cl$Mn_deff`.

³⁹This function is part of the tidyverse language in R, which is what the `dplyr` and `srvyr` packages use.

We first have to prepare the data. Let's pretend we had a *population* of 100 students within each school from which we had randomly sampled. It is super unlikely that we'd ever have a data set with the same population size in all of the schools. This is only a demonstration—in our actual schools, we will probably know the total number in each school. There are big schools and small schools.

```
dat$N_cl_s <- 100
```

Now, we can use this column in the data set.

```
des_cl2 <- dat %>%
  as_survey_design(id = c(school, sid),
                  fpc = c(N_cl, N_cl_s))
summary(des_cl2)
```

```
## 2 - level Cluster Sampling design
## With (8, 32) clusters.
## Called via srvyr
## Probabilities:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 3.2e-07 3.2e-07 3.2e-07 3.2e-07 3.2e-07 3.2e-07
## Population size (PSUs): 1e+06
## Data variables:
## [1] "sid"      "school"    "district"  "score"     "weight"    "N_cl"      "N_cl_s"
```

```
rslts_cl2 <- des_cl2 %>%
  summarize(Mn = survey_mean(score,
                             vartype = c("se", "ci"), df = degf(.),
                             deff = TRUE ),
            Var= survey_var(score) )
rslts_cl2 <- rslts_cl2 %>% mutate(EffSS = n/Mn_deff)
print.data.frame(round(rslts_cl2,2))
```

```
##      Mn Mn_se Mn_low Mn_upp Mn_deff      Var Var_se EffSS
## 1 40.77  3.66  32.12  49.42    2.31 185.28  40.72 13.84
```

The results are not that different from the single-stage cluster-sample design we just analyzed. Again, this is an artificial data set and we would likely specify `N_cl_s` for each person within a school the same but different across schools (of different sizes).

7.5 Stratified design

The strata are districts. In this sampling design, we are randomly selecting students from each stratum and pretending that the school cluster did not exist when we designed and carried out this sampling plan.

7.5.1 Preparing our data set

Again, we'll effectively make the finite population correction negligible and assume that the population is big. This yields a more conservative confidence interval than if we had specified a smaller finite population, which would likely be the case if we were sampling from districts that were small.

Here, our primary sampling unit is the lowest element of our sampling plan. In other words, we had sampled the students from the strata. In the data set, each student is a row.

We need to specify how many sampled units are in each stratum. In other words, how many students are in each district. Normally, there will be a different number of students in a district (some districts are large; some are small). Here, for convenience (and to replicate the results in Heeringa and colleagues' chapter), we will make them all very large, maybe something like 25,000,000 students in each district. In the column in the data, we can think of the students as coming from their district of this population size.

Traditionally, h stands for stratum in research designs. So, we'll call our object `des_h` to help us remember it is a stratified design.

```
dat$N_h = 25000000
```

7.5.2 Step 1: Specifying the design

Notice that we have three arguments: `id = 1` to specify that the primary sampling unit is each row in the data set.⁴⁰ The `strata = district` specifies the stratum. The `fpc = N_h` specifies the column in our data that includes the population size of each stratum.

```
des_h <- dat %>%
  as_survey_design(id = 1,
                  strata = district,
                  fpc = N_h)
summary(des_h)

## Stratified Independent Sampling design
## Called via srvyr
## Probabilities:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 3.2e-07 3.2e-07 3.2e-07 3.2e-07 3.2e-07 3.2e-07
## Stratum Sizes:
##           1 2 3 4
## obs           8 8 8 8
## design.PSU 8 8 8 8
## actual.PSU 8 8 8 8
## Population stratum sizes (PSUs):
##           1         2         3         4
## 2.5e+07 2.5e+07 2.5e+07 2.5e+07
## Data variables:
## [1] "sid"      "school"    "district"  "score"     "weight"    "N_cl"      "N_cl_s"
## [8] "N_h"
```

Again, in our particular example, we're saying each stratum has the same number of students but if we had a different population size in each stratum, we could specify `data$N_h` to be that number within each stratum. For example, if District 1's population were small, with 1,000 students, all the students in our sample would have `N_h = 1000`; if District 2 were larger, with 20,000 students, the students in this district in our sample have `N_h = 20000`.

⁴⁰The `srvyr` and `survey` packages still include an argument for the PSU (`id =`), and instead of clusters as was done with cluster-sampling, we specify `id = 1`, where 1 tells R to treat the individual cases as the PSU. If you use the `survey` package directly, instead of the `srvyr` package as a wrap around, it is `id = ~1`, with the tilde added.

7.5.3 Step 2: Calculating the estimates

This step is identical to what we did before with the other example designs above. The difference is the design itself, which we created in the previous step.

```
rslts_h <- des_h %>%
  summarize(Mn = survey_mean(score,
    vartype = c("se", "ci"), df = degf(.),
    deff = TRUE ),
    Var= survey_var(score) )
rslts_h
```

```
##           Mn    Mn_se  Mn_low  Mn_upp  Mn_deff      Var  Var_se
## 1 40.76875 2.039311 36.59141 44.94609 0.7182879 185.2758 47.70248
```

Here's the effective sample size from n and the design effect. First, let's remember how many observations are in our data:

```
n
```

```
## [1] 32
```

```
rslts_h <- rslts_h %>% mutate(EffSS = n/Mn_deff)
rslts_h
```

```
##           Mn    Mn_se  Mn_low  Mn_upp  Mn_deff      Var  Var_se  EffSS
## 1 40.76875 2.039311 36.59141 44.94609 0.7182879 185.2758 47.70248 44.55038
```

As an extra step, we could report the mean per stratum:

```
M_by_dist <- des_h %>%
  group_by(district) %>%
  summarize(Mn = survey_mean(score),
    Var= survey_var(score))
M_by_dist
```

```
## # A tibble: 4 x 5
##   district    Mn Mn_se    Var Var_se
##   <int> <dbl> <dbl> <dbl> <dbl>
## 1      1  51.1  4.32  150.   67.4
## 2      2  43.3  4.63  172.   73.0
## 3      3  39.6  4.28  146.   84.8
## 4      4  29.0  2.84  64.6   44.4
```

7.6 Stratified cluster-sampled design

In this design, the clusters were randomly sampled from each stratum. In this artificial data set, there were four districts and from each district, two schools were randomly selected from the sampling frame that contained the list of all schools in the population (by district).

7.6.1 Preparing the data

In the data set, we need to specify the number of clusters per stratum. For this example, to make it consistent with what Heeringa and colleagues present in their chapter, we set the population as very high. This results in a negligible (tiny) finite population correction. Let's use something like $n/N = 0.000008$ for each stratum, which would occur if we had $n = 2$ schools in each stratum's sample and $N = 250,000$ schools in its population.

```
dat$N_hcl <- 250000
```

So, we have a column in our data that we will use to say that there are 250,000 schools in each stratum.⁴¹

7.6.2 Step 1: Specifying the design

The design step is similar to what we used in the other cluster-sampled and stratified designs. Here, the variable used as the primary sampling unit is school, which is what `id = school` refers to. The stratum variable in the data set is district, `strata = district`, and the population size of each stratum is the column in the data we just created, `fpc = N_hcl`.

```
des_hcl <- dat %>%
  as_survey_design(id = school,
                  strata = district,
                  fpc = N_hcl)
summary(des_hcl)
```



```
## Stratified 1 - level Cluster Sampling design
## With (8) clusters.
## Called via srvyr
## Probabilities:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 8e-06  8e-06   8e-06   8e-06  8e-06   8e-06
## Stratum Sizes:
##           1 2 3 4
## obs           8 8 8 8
## design.PSU 2 2 2 2
## actual.PSU 2 2 2 2
## Population stratum sizes (PSUs):
##           1      2      3      4
## 250000 250000 250000 250000
## Data variables:
## [1] "sid"      "school"    "district"  "score"     "weight"    "N_cl"      "N_cl_s"
## [8] "N_h"      "N_hcl"
```

We can see the sample sizes of the strata and the primary sampling unit (PSU) per stratum. This very simple example has only two clusters (schools) within each stratum (district).

7.6.3 Step 2: Calculating the estimates

This step is the same as the other designs above. We can also tag on the effective sample size.

⁴¹Obviously, that is a lot of schools to have in a district. This does make the error more conservatively estimated. Also, this seems to be in the spirit of model-based inference, where the population is more theoretical.

```

rslts_hcl <- des_hcl %>%
  summarize(Mn = survey_mean(score,
    vartype = c("se", "ci"), df = degf(.),
    deff = TRUE ),
    Var= survey_var(score) )
# effective sample size
n <- length(unique(dat$cid))
rslts_hcl <- rslts_hcl %>% mutate(EffSS = n/Mn_deff)
rslts_hcl

```

```

##           Mn      Mn_se  Mn_low  Mn_upp  Mn_deff      Var  Var_se  EffSS
## 1 40.76875  2.769807 33.07853 48.45897  1.325055 185.2758 49.58831 24.14994

```

```

#-----
# Extra step:
# We can look at the mean per stratum :

M_by_Dist_cl <- des_hcl %>%
  group_by(district) %>%
  summarize(Mn = survey_mean(score),
    Var= survey_var(score))
M_by_Dist_cl

```

```

## # A tibble: 4 x 5
##   district      Mn Mn_se   Var Var_se
##   <int> <dbl> <dbl> <dbl> <dbl>
## 1      1  51.1  3.89 150.   36.3
## 2      2  43.3  8.91 172.   48.7
## 3      3  39.6  3.95 146.   57.5
## 4      4  29.0  3.55  64.6  35.5

```

7.7 A simple random sample design, for comparison

The simple random sample design assumes that the students were randomly drawn from a population of students. There is no clustering or stratification in the design or analysis.

7.7.1 Preparing our data set

We can, of course, simply use Base R functions to estimate the mean and variance and then calculate the standard error as $\sqrt{\frac{Var}{n}}$ and then use the calculations we used in the prior chapter. However, the `srvyr` package also allows us to use the finite population correction.

If we assumed the population were huge, our finite population correction would be negligible. Let's do this:

```

n <- length(unique(dat$cid)) # sample n
dat$N <- 10^8 # Population N
des_srs <- dat %>%
  as_survey_design(id = 1, fpc = N)
rslts_srs <- des_srs %>%
  summarize(Mn = survey_mean(score, vartype = c("se", "ci"), df = degf(.)),
    Var= survey_var(score))

```

```

rslts_srs <- rslts_srs %>%
  mutate(EffSS = n,
         Mn_deff = 1)
print.data.frame( round(rslts_srs, 2) )

```

```

##      Mn Mn_se Mn_low Mn_upp   Var Var_se EffSS Mn_deff
## 1 40.77  2.41  35.86  45.68 185.28  45.96    32      1

```

Notice that the results are similar to the results we'd get using the procedures in the previous chapter.

```

score_M <- mean(dat$score)
score_sd <- sd(dat$score)
score_n <- nrow(dat)
score_se <- score_sd/sqrt(score_n)
LB_t <- score_M - qt(0.975, df = score_n - 1)*score_se
UB_t <- score_M + qt(0.975, df = score_n - 1)*score_se
round((c(M = score_M, var = score_sd^2, n = score_n, se = score_se,
        CI_upper = LB_t, CI_lower = UB_t)), 2)

```

```

##      M      var      n      se CI_upper CI_lower
## 40.77 185.28  32.00  2.41  35.86  45.68

```

If our finite population were small, such as $N = 100$, our standard error would be smaller and our confidence interval would be narrower, indicating our estimate of the mean is a more precise measure of a population of size 100.

```

dat$N2 <- 100 # Small Population N
des_srs2 <- dat %>% as_survey_design(id = 1, fpc = N2)
rslts_srs2 <- des_srs %>%
  summarize(Mn = survey_mean(score, vartype = c("se", "ci"), df = degf(.)))
print.data.frame( round(rslts_srs2, 2) )

```

```

##      Mn Mn_se Mn_low Mn_upp
## 1 40.77  2.41  35.86  45.68

```

So, even though we can use Base R to estimate the confidence interval of a simple random sample, the survey and srvyr packages allow us to account for the finite population correction.

7.7.2 Comparing all design scenarios

Note that we would **not** do this in practice. In practice, we would select a single design and report the results. This comparison is only for seeing how the designs operate.

```

##      Design      Mn Mn_se Mn_low Mn_upp Mn_deff EffSS   Var Var_se
## 1 Simple Rand. Samp. 40.77  2.41  35.86  45.68   1.00 32.00 185.28  45.96
## 2      Clustered 40.77  3.66  32.12  49.42   2.31 13.84 185.28  40.72
## 3 two-stage Clustered 40.77  3.66  32.12  49.42   2.31 13.84 185.28  40.72
## 4      Stratified 40.77  2.04  36.59  44.95   0.72 44.55 185.28  47.70
## 5 Stratified Clustered 40.77  2.77  33.08  48.46   1.33 24.15 185.28  49.59

```

Compare this to Table 2.2 in Heeringa, et al. [2010]. The small differences (e.g., in the effective n-size) are due to rounding error in their chapter.

Of these designs, which seems to have the most precise estimate of the mean of the population? Which seems to provide the best balance between precision and feasibility in carrying out the sampling process?

Updated on 2021-04-21.

References

- Greg Freedman Ellis and Ben Schneider. *srvyr: dplyr-Like Syntax for Summary Statistics of Survey Data*, 2021. URL <https://CRAN.R-project.org/package=srvyr>. R package version 1.0.1.
- Steven G. Heeringa, Brady T. West, and Patricia A. Berglund. *Applied survey data analysis*. Chapman & Hall/CRC, Boca Raton, FL, 2010.
- Thomas Lumley. *survey: Analysis of Complex Survey Samples*, 2020. URL <http://r-survey.r-forge.r-project.org/survey/>. R package version 4.0.
- William Revelle. *psych: Procedures for Psychological, Psychometric, and Personality Research*, 2020. URL <https://personality-project.org/r/psych/><https://personality-project.org/r/psych-manual.pdf>. R package version 2.0.9.
- Hadley Wickham, Winston Chang, Lionel Henry, Thomas Lin Pedersen, Kohske Takahashi, Claus Wilke, Kara Woo, Hiroaki Yutani, and Dewey Dunnington. *ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*, 2020a. URL <https://CRAN.R-project.org/package=ggplot2>. R package version 3.3.2.
- Hadley Wickham, Romain François, Lionel Henry, and Kirill Müller. *dplyr: A Grammar of Data Manipulation*, 2020b. URL <https://CRAN.R-project.org/package=dplyr>. R package version 1.0.2.